BAN404 compendium

Table of Contents

Chapter 1: Introduction 4
Notation 4
Chapter 2: Statistical learning 4
What is statistical learning? 4
Why do we estimate f? 4
How do we estimate f? 5
The trade-off between prediction accuracy and model interpretability 6
Supervised vs. unsupervised learning 6
Regression vs. classification problems 6
Assessing model accuracy 7
Measuring the quality of fit 7
The bias variance trade-off 7
The classification setting 8
Chapter 3: Linear Regression 9
Simple linear regression 9
Estimating the coefficients 10
Assessing the accuracy of the coefficient estimates 10
Assessing the accuracy of the model 11
Multiple linear regression 11
Estimating the regression coefficients 11
Some important questions 11
Other considerations in the regression model 12
Extensions of the linear model 12
Potential problems 12
Comparison of linear regression with K-nearest neighbors 12
Chapter 4: Classification 13
An overview of classification 13
Why not linear regression? 13
Logistic regression 13
The logistic model 13
Estimating the regression coefficients 14

Making predictions 14
Multiple logistic regression 15
Multinomial logistic regression 15
Generative models for classification 15
Linear discriminant analysis for p = 1 16
Linear discriminant analysis for p > 1 16
Quadratic discriminant analysis 17
Naïve Bayes 17
A comparison of classification models 18
Chapter 5: Resampling methods 18
Cross-validation 18
The validation set approach 19
Leave-One-Out Cross-Validation 19
k-Fold Cross-Validation 19
Bias-variance trade-off for k-Fold Cross-Validation 20
Cross-Validation on classification problems 21
The Bootstrap 21
Chapter 6: Linear Model Selection and Regularization 21
Subset selection 22
Best subset selection 22
Stepwise selection 23
Choosing the optimal model 24
Shrinkage methods 25
Ridge regression 25
The lasso 25
Dimension reduction methods 26
Principal components regression 27
Chapter 7: Moving Beyond Linearity 27
Polynomial regression 27
Step functions 27
Basis functions 28
Regression splines 28
Piecewise polynomials 28
Choosing the number and locations of the knots 29
Comparison to polynomial regression 29

Smoothing splines 29
Choosing the smoothing parameter λ 30
Local regression 30
Generalized Additive Models 31
GAMs for regression problems 31
Pros and cons of GAMs 32
Chapter 8: Tree-Based Methods 33
Regression Trees 33
Classification trees 34
Advantages and disadvantages of trees 35
Bagging, Random Forests, Boosting, and Bayesian Additive Regression Trees 35
Bagging 36
Random forests 37
Boosting 38
Summary of tree ensemble methods 39
Chapter 9: Support Vector Machines 39
Maximal margin classifier 39
What is a hyperplane? 39
Classification using a separating hyperplane 40
The maximal margin classifier 40
The non-separable case 41
Support vector classifiers 41
Details of the support vector classifier 41
Support vector machines 43
Classification with non-linear decision boundaries 43
The support vector machine 44
Chapter 12: Unsupervised learning 45
The challenge of unsupervised learning 45
Principal components analysis 45
What are principal components? 45
Clustering methods 47
K-means clustering 48
Hierarchical clustering 49

Chapter 1: Introduction

Statistical learning refers to a vast set of tools for understanding data. The tools themselves can be classified as supervised or unsupervised.

Supervised learning: Building a statistical model for predicting or estimating an output based on one or more inputs.

Unsupervised learning: with unsupervised statistical learning there are inputs but no supervising output. Nevertheless, we can learn relationships and structures from such data.

Notation

Symbol	Description
n	Number of distinct data points (observations)
p	Number of variables available for use
$x_{i,j}$	Value of the jth variable for the ith observation

Chapter 2: Statistical learning

What is statistical learning?

Generally, we want to predict how some predictors affect our output variable. The general equation will look like this:

$$Y = f(X) + \varepsilon$$

Here, f is a fixed, yet unknown, function of X_1, ..., X_p, and where ε is a random error term which is independent of X and has a mean of zero. In this representation, f represents the systematic information that X provides about Y.

In essence, statistical learning refers to a set of approaches for estimating f.

Why do we estimate f?

There are two main reasons: Prediction and inference.

Prediction

Relevant where a set of inputs X are readily available, but the output variable cannot easily be obtained. Because the error term averages to zero, we can predict Y using:

$$\hat{Y} = \hat{f}(X)$$

Here, $\hat{f}$ is our estimate for f, and $\hat{Y}$ represents the resulting prediction for Y. The accuracy of $\hat{Y}$ as a prediction for Y depends on two factors: the reducible error and the irreducible error. In general, $\hat{f}$ is not a perfect estimate of f, and this imperfection introduces error. This error is reducible because we can potentially improve $\hat{f}$'s accuracy by employing the most suitable statistical learning technique to estimate f.

However, even if we could achieve a perfect estimate of f, such that our predicted response took the form $\hat{Y} = \hat{f}(X)$, there would still be some error in our prediction. This occurs because Y also depends on ε which, by definition, cannot be predicted using X.

Consequently, the variability associated with ε affects the accuracy of our predictions. This is referred to as the irreducible error, as no matter how well we estimate f, we cannot eliminate the error introduced by ε.

The aim of this course is to use techniques for minimizing the reducible error.

Inference

In the instance of inference, we still want to estimate f, but the goal here is not necessarily to make predictions for Y. I contrast to prediction; we now actually care about the form of $\hat{f}$ which in the previous case acted as a black box. One may be interested in answering these questions:

- Which predictors are associated with the response?
- What is the relationship between the response and each predictor?
- Can the relationship between Y and each predictor be adequately summarized using a linear equation, or is the relationship more complicated?

How do we estimate f?

Although we explore many methods for estimating f, they generally share certain characteristics.

Training data: Data used to train our method how to estimate our f

There are generally two approaches: parametric and non-parametric.

Parametric methods

Involves a two-step model-based approach:

1. First, we assume the functional form of the model
2. After a model has been selected, we need a procedure that uses the training data to fit or train the model.
    a. An example is estimating parameters
    b. The most common approach is referred to as ordinary least squares (OLS)

The model-based approach described is referred to as parametric. It reduces the problem down to estimating a set of parameters. A disadvantage of this approach is that the model chosen usually does not match the true unknown form of f. If the chosen model is too far away, the estimate will be poor.

The more parameters you add to the model, making it more complex, can lead to overfitting. This phenomenon says that the model follows the errors, or noise, too closely.

Non-parametric methods

These methods do not make explicit assumptions about the functional form of f. Instead, they seek an estimate of f that gets as close to the data points as possible without being too rough or wiggly.

These methods have an advantage compared to parametric methods by being able to fit a wider range of possible shapes for f. On the other hand, non-parametric methods have a disadvantage in needing far more observations for an accurate estimate because parameters are not estimated.

The trade-off between prediction accuracy and model interpretability

Flexibility refers to how many potential shapes a model has the potential to make in the estimation of f.

FIGURE 2.7. *A representation of the tradeoff between flexibility and interpretability, using different statistical learning methods. In general, as the flexibility of a method increases, its interpretability decreases.*

But low flexibility seems negative compared to high flexibility. Why would then anyone ever choose a more restrictive method instead of a more flexible one? We now meet the concept of interpretability.

For instance, when inference is the goal, the linear model may be a good choice since it will be quite easy to understand the relationship between Y and $X_1, X_2, \dots, X_p$. In contrast, very flexible approaches can lead to such complicated estimates that they make it difficult to understand how any individual predictor is associated with the response.

It is important to mention that in other examples prediction is the only thing that matters, and that the interpretability of the model is not of any interest. Although it may seem that more flexible models create better predictions, this is not always the case because of overfitting.

Supervised vs. unsupervised learning

Supervised learning is in many ways easier to grasp because there are both predictors and a response variable. With unsupervised learning, we are more or less working blind; we lack a response variable that can supervise our analysis. Cluster analysis is a clear example of unsupervised learning, where the model is supposed to tell whether an observation falls within a certain category.

Regression vs. classification problems

Variables can be either quantitative or qualitative (categorical). We refer to problems with quantitative variables as regression problems, while classification problems consist of qualitative variables. We tend to select statistical learning methods based on whether the response is quantitative or qualitative. Whether the predictors are qualitative or quantitative are of less importance.

Assessing model accuracy

The need for introducing a plethora of methods for statistical learning comes from there not being a single best method for every use. No one method dominates all other methods over all data sets. Therefore, selecting the best method for the given data is essential. This is not easy, and the topics discussed below are therefore extremely important.

Measuring the quality of fit

We need a way to measure how well a model's predictions match the observed data. This needs to be done for every single observation. In the context of regression, the most used measure is the mean squared error (MSE), given by

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2$$

Where $\hat{f}(x_i)$ is the prediction that $\hat{f}$ gives for the ith observation. The MSE will be small if the predicted responses are very close to the true responses and will be large if for some of the observations, the predicted and true responses differ substantially.

Even though we may use the MSE measure to gauge the fit on the training data, the only thing we really care about is whether the MSE measure is good for the unseen test data. Why? Well, because we already have the response of the training data, but the goal of our approach is to be able to predict future responses. Therefore, we always want to choose the model with the lowest test MSE rather than training MSE.

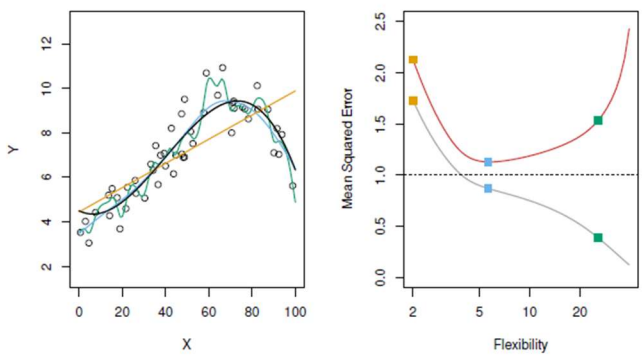
When there are no test observations, one may think that the sensible course of action is to choose the model with the lowest training MSE. The unfortunately is a fundamental problem with this strategy. There is no guarantee that the lowest training MSE method will have the lowest test MSE.

When developing a model, the more degrees of freedom, the lower MSE for the training data. But as we know from earlier too many degrees of freedom will lead to overfitting because the model will pick up effects that are created by chance.

Lastly, we almost always will expect training MSE to be lower than test MSE. This makes sense because the test data is completely new for the model, and patterns that have occurred in the training data may not exist in the test data and vice versa.

The bias variance trade-off

For different models in the book, we observe a U-shape for the observed test MSE's with degree of flexibility on the x-axis and MSE on the Y-axis.



You can see an example above. Mathematics beyond the scope of this course can prove that expected MSE for a value x_0 can be decomposed into three fundamental quantities: The variance of $\hat{f}(x_0)$, the squared bias of $\hat{f}(x_0)$ and the variance of the error terms ε.

In essence, we need to select a statistical learning method that simultaneously achieves low variance and low bias to minimize the expected error. Note that the expected error never lays below Var(ε), the irreducible error, since it always is non-negative. Then, what do we mean when we talk about bias and variance?

Variance: The amount of which $\hat{f}$ would change if we estimated it using a different training data set. Ideally $\hat{f}$ should not change much, but with high variance small changes in training data could result in big changes in $\hat{f}$. Generally, more flexible statistical methods have higher variance.

Bias: The error that is introduced by approximating a real-life problem, which may be complicated, by a much simpler model. For example, by using a linear regression model one implies a linear relationship between the predictors. This is highly unlikely, seen from a real life perspective. Generally, more flexible methods result in less bias.

Generally, more flexible methods lead to higher variance but lower bias. Whether the test MSE increases or decreases depends on the relative rate of these changes. Initially, as flexibility increases, bias decreases faster than variance grows, reducing the test MSE. However, beyond a certain point, flexibility has little effect on bias but significantly increases variance, causing the test MSE to rise.

As we now understand, there is a trade-off between bias and variance. The challenge lies in finding a method in which both variance and squared bias are low.

The classification setting

So far, the model accuracy aspect has mostly been discussed within the context of regression. But many of the concepts that have been mentioned also transfer over to the classification setting. Seeing that y_i is no longer quantitative, we need some modifications.

The most common approach for quantifying the accuracy of our estimate for classification problems is the training error rate. This number tells us the proportion of mistakes that are made if we apply our estimate to the training observations:

$$\frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}_i)$$

Here $\hat{y}_i$ is the predicted class label for the ith observation using $\hat{f}$. And $I(y_i \neq \hat{y}_i)$ is an indicator variable that equals 1 if $y_i \neq \hat{y}_i$ and zero if $y_i = \hat{y}_i$. If $I(y_i \neq \hat{y}_i)$ = 0 then the ith observation was classified correctly by our variable classification method; otherwise, it was misclassified.

But as we remember we are most interested in the application on the test data. This is called the test error rate.

$$Ave(I(y_i \neq \hat{y}_i))$$

A good classifier is one for which the test error is smallest.

The Bayes classifier

It is mathematically provable that error rate is minimized with a simple classifier that assigns each observation to the most likely class, given its predictor values. In other words, we should simply assign a test observation with predictor vector x_0 to the class j for which

$$\Pr\,(Y = j|X = x_0)$$

is largest. This is a conditional probability that is called the Bayes classifier. Generally, if the observation entered into the model is most likely to be a given value, the classifier will assign the observation said value.

K-nearest neighbors

As we often do not know the distribution of the observations we cannot use the Bayes classifier. Another approach is to look at the nearest observations (neighbors) to an observation and this way assign the class with the highest estimated probability.

$$\Pr(Y = j|X = x_0) = \frac{1}{K}\sum_{i \in N_o} I(y_i = j)$$

The choice of K is essential in this approach. A K of 1 is vastly overfit (too flexible) finding patterns in the data that is just noise. As K grows the method becomes less flexible and produces a decision boundary that is close to linear. There are methods to find the optimal K.

To conclude, choosing the correct level of flexibility of the model is critical to the success of any statistical learning method. The bias-variance trade-off, and the resulting U-shape in the test error, can make this a difficult task.

Chapter 3: Linear Regression

Linear regression is a simple, but useful, tool for predicting a quantitative response. It also serves as a good starting point for other more advanced methods that we are covering. Linear regression can answer some of these questions related to the advertising data set:

- Is there a relationship between advertising budget and sales?
- How strong is the relationship between advertising budget and sales?
- Which media are associated with sales?
- How large is the association between each medium and sales?
- How accurately can we predict future sales?
- Is the relationship linear?
- Is there synergy among the advertising media?

Simple linear regression

Simple linear regression assumes that there is a linear relationship between Y and a single predictor variable X. Mathematically, we can write this linear relationship as

$$Y \approx \beta_0 + \beta_1 X$$

In this equation there are two unknown constants that represent the intercept and slope terms of the linear model. Together, these are known to be model coefficients or

parameters. Once we have used our training data to estimate model coefficients, we can predict future Y based on a particular value of X by computing

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 X$$

The hat symbolizes the estimated coefficients and predictions for the response.

Estimating the coefficients

To get the most accurate predictions, we need to minimize the total distance between the regression line and all the data points. Specifically, the most common approach involves minimizing the least squares criterion. Using the same predicted equation as above, we have a residual for each data point, given as

$$e_i = y_i - \hat{y}_i$$

This is the difference between the ith observed response value that is predicted by our linear model. We define the residual sum of squares (RSS) as

$$RSS = e_1^2 + e_2^2 + \cdots + e_n^2$$

The least squares approach chooses $\hat{\beta}_0$ and $\hat{\beta}_1$ to minimize the RSS. Using some calculus, one can show that the minimizers are

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2},$$
$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x},$$

Assessing the accuracy of the coefficient estimates

Thus far we have forgotten about the error term ε that is a catch-all for what we miss with our estimated model. In reality, there are almost never any linear relationships. The relationship can now be correctly formulated as

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

Some important quantities:

Term	**Description**
Coefficient	The estimated effect of a predictor (independent variable) on the response (dependent variable) in a model. For example, it tells us how much the response changes when the predictor increases by one unit.
Standard error	The accuracy of the coefficient estimate. A smaller standard error means the estimate is more precise, while a larger one suggests more uncertainty.
t-statistic	The ratio of the coefficient to its standard error. It helps test if the coefficient is significantly different from zero (i.e., whether the predictor has a real effect).
p-value	The probability of observing the given result (or something more extreme) if the predictor has no real effect. A smaller p-value (typically less than 0.05) suggests the predictor is statistically significant.

Assessing the accuracy of the model

Once we have rejected the null hypothesis in favor of the alternative hypothesis, it is natural to want to quantify to which extent the model fits the data. The quality of a linear regression model is typically assessed using two related quantities: residual standard error (RSE) and the R^2 statistic. We explain the terms briefly below.

Term	Description
Residual standard error (RSE)	This measures the typical size of the errors made by the model when predicting the response. In other words, the estimated standard deviation of ε. A smaller RSE means the model's predictions are closer to the actual data.
R^2	This shows how well the model explains the variation in the response. It ranges from 0 to 1, with higher values indicating that the model explains more of the variation in the data. Considering RSE is measured in unit of Y, it can sometimes be unclear. This measure can be more interpretable as it is expressed as a percentage.
F-statistic	This tests whether the overall model is useful by comparing it to a model with no predictors. A higher F-statistic means the predictors significantly improve the model's ability to explain the response.

Multiple linear regression

Simple linear regression is often unrealistic as we in practice often have more than one predictor variable. It may also be foolish to fit simple linear regression models for each individual predictor, so we need a method for fitting them together to see interaction between the variables. To accommodate this, we extend the original regression model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon$$

We now have slope coefficients for each of the predictors.

Estimating the regression coefficients

The parameters are still estimated using the least squares approach, e.g. we want to minimize the sum of squared residuals (RSS). The results of a multiple regression model compared to a simple regression model for one of the parameters can look very different. Where a predictor can be significant in a simple regression model, it can not be in a multiple regression model. It is also important to note correlations between the predictors and the effects that may come of this.

Some important questions

- Is at least one of the predictors useful in predicting the response?
- Do all of the predictors help to explain Y, or is only a subset of the predictors useful
- How well does the model fit the data?
- Given a set of predictor values, what response value should we predict, and how accurate is our prediction?

Other considerations in the regression model

Qualitative predictor with two levels: One dummy

Qualitative predictor with multiple levels: One dummy for each level, binary

Extensions of the linear model

A linear model makes two highly restrictive assumptions: The relationship between output and predictors is additive and linear. Additive means that the association between a predictor and output does not depend on the values of other predictors. Linear means that the change in the response associated with a one-unit change in the predictor Is constant. These assumptions are rarely realistic, and we therefore want to relax these assumptions in later models.

Removing additive assumption

Predictors can have synergy (interaction) effects, meaning that predictors together shape the response. Compared to a standard linear regression, we can add a new interaction term called the interaction term:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \varepsilon$$

We can interpret the intercept coefficient as the increased effectiveness of one predictor associated with a one-unit increase in the other predictor. In the case of where the interaction term is significant, but the predictors individually are not, we should include the predictors to the model anyways according to the hierarchical principle.

Non-linear relationships

The true relationship between output and predictor may be non-linear. A way to attack this is with polynomial regression. We can add multiple polynomials of the same predictors to a model, and sometimes they all seem significant. Are there any problems related to this?

Potential problems

When we fit a linear regression model to a particular data set, many problems may occur. Most common among these are the following:

- Non-linearity of the response-predictor relationships
- Correlation of error terms
- Non-constant variance of error terms
- Outliers
- High-leverage points
- Collinearity

Comparison of linear regression with K-nearest neighbors

Linear regression is as mentioned an example of a parametric approach, assuming a linear functional form for $f(X)$. Assuming the functional form can be a disadvantage if the estimated model is far from the truth, and model accuracy is the goal.

In contrast, non-parametric methods do not explicitly assume a parametric form for $f(X)$, being a more flexible approach for performing regression. Maybe the simplest of these approaches is KNN regression. The optimal value for K depends on the bias-variance tradeoff.

Even though the non-parametric approaches may seem better most of the time, parametric approaches could outperform non-parametric approaches if the functional form selected is close to the true form of f. Also, when the number of dimensions increases, KNN can give worse results given the curse of dimensionality.

Chapter 4: Classification

In contrast to linear regression, the output in a classification problem is qualitative. Both qualitative and categorical are terms that are interchangeably used to describe this type of output variable. Classification refers to predicting a categorical (qualitative) response. In this chapter we are discussing some widely used classifiers: Logistic regression, linear discriminant analysis, quadratic discriminant analysis, naïve Bayes, and K-nearest neighbors.

An overview of classification

Some examples of classification problems include:

- A person arrives at the emergency room with a set of symptoms that could possibly be attributed to one of three medical conditions. Which of the three conditions does the individual have?

- An online banking service must be able to determine whether or not a transaction being performed on the site is fraudulent, on the basis of the user's IP address, past transaction history, and so forth.

- On the basis of DNA sequence data for a number of patients with and without a given disease, a biologist would like to figure out which DNA mutations are deleterious (disease-causing) and which are not.

In the exact way we approach a regression problem, we use training data to fit a classifier that hopefully will, as correctly as possible, predict test data.

Why not linear regression?

If the output variable of a classification problem is continuous, we imply an ordered relationship between the outputs, something that is quite rare. For problems with more than two responses, there would need to be a very logical natural ordering for a linear regression to give meaningful output.

Logistic regression

The example we are covering in the chapter is whether or not an individual will default on their loan. In other words, the response falls into one of two categories: Yes or no. Instead of modelling this response directly, logistic regression models the probability that Y belongs to a particular category. One can choose which level of probability is needed to fall into a certain category. Sometimes it is 0.5, but the more conservative one wishes to be, the lower you can set the bar for a default for instance.

The logistic model

The probability of a certain outcome in logistic regression is modelled like so:

$$p(X) = \Pr(Y = 1|X)$$

Because we need to model probability, we need $p(X)$ to always lay between 0 and 1. Therefore we cannot use a regular linear regression. In logistic regression we use a logistic function to combat the problem:

$$p(X) = \frac{e^{\beta_0+\beta_1 X}}{1 + e^{\beta_0+\beta_1 X}}$$

To fit the model, we use a method called maximum likelihood, which we discuss in the next section. For low values of X, we now predict the probability of $Y = 1$ to be close to, but never equal to 0. For high values of X we predict the opposite. Thus, the logistic function will always produce a S-shaped curve.

We can manipulate the expression to indicate the odds:

$$\frac{p(X)}{1 - p(X)} = e^{\beta_0+\beta_1 X}$$

Values of odds close to 0 and ∞ indicate very low and very high probabilities of default, respectively. Odds are traditionally used instead of probabilities in horse-racing, since they relate more naturally to the correct betting strategy. By taking the logarithm of each side, we arrive at:

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X$$

The left side is now called the log odds, or logit. We see that the logistic regression model has a logit that is linear in X. Recall from Chapter 3 that in a linear regression model, β_1 gives the average change in Y associated with a one-unit increase in X. By contrast, in a logistic regression model, increasing X by one unit changes the log odds by β_1. Equivalently, it multiplies the odds by e^{β_1}. However, because the relationship between $p(X)$ and X is not a straight line, β_1 does not correspond to the change in $p(X)$ associated with a one-unit increase in X. The amount that $p(X)$ changes due to a one-unit change in X depends on the current value of X. But regardless of the value of X, if β_1 is positive then increasing X will be associated with increasing $p(X)$, and if β_1 is negative then increasing X will be associated with decreasing $p(X)$. The fact that there is not a straight-line relationship between $p(X)$ and X, and the fact that the rate of change in $p(X)$ per unit change in X depends on the current value of X.

Estimating the regression coefficients

As mentioned earlier, we use maximum likelihood to estimate the coefficients for its statistical properties. The intuition behind maximum likelihood is to fit a logistic regression model that creates estimates for β_0 and β_1 that corresponds as closely as possible to the observed output. The mathematical detail of maximum likelihood is beyond the scope of this course but can be easily fit using R.

Making predictions

After obtaining model coefficients we can for instance predict the probability of default given a certain credit card balance. If the coefficient for balance is positive, the probability of default increases with a higher credit card balance and vice versa. We can also add dummy

variables to the model, for example indicating whether an observation is for a student or not.

Multiple logistic regression

We now consider the problem of predicting a binary response using multiple predictors. We can generalize the model to be as follows:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

Just as with regular logistic regression, we use maximum likelihood to obtain interpretable regression coefficients that we can evaluate and use to predict the output.

Multinomial logistic regression

Sometimes we wish to classify a response variable that has more than two classes. The logistic regression approach we have covered so far only allows for two classes, creating the need for a new model. To do this, we select a single class to be the baseline; without loss of generality, we select the Kth class for this role. Then, we replace the old logistic regression model with this model:

$$\Pr(Y = k|X = x) = \frac{e^{\beta_{k0}+\beta_{k1}x_1+\cdots+\beta_{kp}x_p}}{1+\sum_{l=1}^{K-1} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}}$$

And

$$\Pr(Y = K|X = x) = \frac{1}{1+\sum_{l=1}^{K-1} e^{\beta_{l0}+\beta_{l1}x_1+\cdots+\beta_{lp}x_p}}$$

It is not hard to show that for k = 1, … , K – 1,

$$\log\left(\frac{Pr(Y = k|X = x)}{\Pr\,(Y = K|X = x}\right) = \beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p$$

The coefficient estimates will differ between the two fitted models due to the differing choice of baseline, but the fitted values (predictions), the log odds between any pair of classes, and the other key model outputs will remain the same. Nonetheless, interpretation of the coefficients in a multinomial logistic regression model must be done with care, since it is tied to the choice of baseline.

Generative models for classification

Instead of modelling the conditional distribution of the response Y, given the predictor(s) X, we have a less direct approach to estimating these probabilities. In the new approach, we model the distribution of the predictors X separately in each of the response classes (i.e. for each value of Y). Bayes' theorem is then used to flip these around into estimates for $\Pr(Y = k|X = x)$. When the distribution of X within each class is assumed to be normal, it turns out that the model is very similar in form to logistic regression.

Then, why do we need another method when we have logistic regression? Here are some reasons for why:

- When there is substantial separation between the two classes, the parameter estimates for the logistic regression model are surprisingly unstable. The methods that we consider in this section do not suffer from this problem.

- If the distribution of the predictors X is approximately normal in each of the classes and the sample size is small, then the approaches in this section may be more accurate than logistic regression.

- The methods in this section can be naturally extended to the case of more than two response classes. (In the case of more than two 142 4. Classification response classes, we can also use multinomial logistic regression from Section 4.3.5.)

Suppose that we wish to classify an observation into one of K classes, where K ≥ 2. In other words, the qualitative response variable Y can take on K possible distinct and unordered values. Let π_k represent the overall prior probability that a randomly chosen observation comes from the kth class. Let $f_k(X) \equiv \Pr(X|Y = k)^1$ denote the density function of X for an observation that comes from the kth class. In other words, $f_k(x)$ is relatively large if there is a high probability that an observation in the kth class has $X \approx x$, and $f_k(x)$ is small if it is very unlikely that an observation in the kth class has $X \approx x$. Then Bayes' theorem states that

$$\Pr(Y = k|X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^{K} \pi_l f_l(x)}$$

In accordance with our earlier notation, we will use the abbreviation $p_k(X) = \Pr\,(Y = k|X = x)$. This is the posterior probability that an observation $X = x$ belongs to the kth class.

Linear discriminant analysis for p = 1

Linear Discriminant Analysis (LDA) is well-established in statistical learning for classification problems, in which a categorical response variable can take one of K possible classes. Given the features of a new data point, LDA calculates the probability of that point belonging to each class by assuming normality of the features within each class and uses Bayes' rule to choose the class with the highest such probability. In particular, LDA generally assumes that the feature vectors for each class have a Gaussian distribution with class-specific mean but a common covariance matrix across all classes.

For statistical learning, it mostly serves as a discriminative classifier to provide linear decision boundary LDA is favoured over more flexible techniques when the number of training observations is small, compared to the number of predictors, due to its modelling of the data distribution, which can result in a less flexible lower variance model. It has long been employed for the classification of qualitative values.

As an example, let's say you want to use the frequency of some words (predictors) to determine if an Email is either "Spam" or "Not Spam". LDA would capture the distribution of word frequencies for the emails, which are known to be Spam or non spam, provided that it is Gaussian and the structure is similar. When a new email comes, LDA determines the probability of it being "Spam" or "Not Spam" based on its word counts added to these pre-computed distributions and then classifies it to the category with the higher probabilities.

Linear discriminant analysis for p > 1

Linear Discriminant Analysis (LDA) employs distinct modeling approaches depending on whether the analysis involves a single predictor ($p = 1$) or multiple predictors ($p > 1$). The key distinction lies in the assumed distribution of the predictors within each class and the corresponding parameters that must be estimated.

In the case of a single predictor ($p = 1$), LDA assumes that the observations within each class are drawn from a univariate normal distribution. Specifically, for the k-th class, the predictor X follows a normal distribution characterized by a class-specific mean μ_k and a common variance σ^2 shared across all K classes. The density function $f_k(x)$ for each class is derived from this univariate normal distribution, and the Bayes classifier assigns observations to the class with the highest estimated probability based on these densities.

When multiple predictors are present ($p > 1$), LDA extends this assumption to a multivariate Gaussian distribution. Here, the predictor vector $X = (X_1, X_2, \dots, X_p)$ for the k-th class is modeled using a multivariate normal distribution with a class-specific mean vector μ_k and a common covariance matrix Σ shared across all classes. Estimating the density function $f_k(x)$ now involves computing the multivariate Gaussian density, which requires estimating significantly more parameters compared to the univariate case. Specifically, the common covariance matrix Σ contains $p(p+1)/2$ unique parameters, making the estimation process more complex as the number of predictors increases.

Despite these differences in dimensionality and parameterization, the resulting classification rule in LDA remains linear in x for both scenarios. The primary challenge in the multivariate case lies in accurately estimating the covariance structure, particularly when the number of predictors is large. Nonetheless, the underlying principle of assigning observations to the class with the highest posterior probability remains consistent across both univariate and multivariate implementations of LDA.

Quadratic discriminant analysis

Like LDA, the QDA classifier results from assuming that the observations from each class are drawn from a Gaussian distribution and plugging estimates for the parameters into Bayes' theorem in order to perform prediction. However, QDA assumes that each class has its own covariance matrix.

The implications of this is bias-variance trade-off when comparing to LDA. With individual parameters, QDA is much more flexible. And at this point, we know the positives and negatives many parameters bring. With few training observations, LDA is often the better bet because reducing variance is crucial. On the other hand, if the training set is very large, so that the variance is not a major concern, QDA is rather recommended.

Naïve Bayes

We recall that Bayes' theorem provides an expression for the posterior probability $p_k(X) = \Pr(Y = k|X = x)$ in terms of $\pi_1, \dots, \pi_k$ and $f_1(x), \dots, f_K(x)$. Estimating the prior probabilities is pretty straightforward. However, estimating $f_1(x), \dots, f_K(x)$ is more subtle. We also recall that $f_K(x)$ is the p-dimensional density function for an observation in the kth class. In the previous models, we have made strong assumptions about this density function. Doing this, we are able to replace the very challenging problem of estimating K p-

dimensional density functions with the much simpler problem of estimating K p-dimensional mean vectors and one or K (p x p)-dimensional covariance matrices.

The naïve Bayes classifier estimates these functions in a different way. Instead of assuming that these functions belong to a particular family of distributions, we instead make a single assumption that: *Within the k-th class, the p predictors are independent*. Stated mathematically, this assumption means that for k = 1, … , K,

$$f_k(x) = f_{k1}(x_1) \times f_{k2}(x_2) \times \ldots \times f_{kp}(x_p)$$

Where f_{kj} is the density function of the jth predictor among observations in the kth class. Why is this assumption so powerful? Essentially, estimating a p-dimensional density function is challenging because we must consider not only the marginal distribution of each predictor – that is, the distribution of each predictor on its own – but also the joint distribution of predictors – that is, the association between different predictors. In the case of a multivariate normal distribution, the association between the different predictors is summarized by the off-diagonal elements of the covariance matrix. However, in general, this association can be very hard to characterize, and exceedingly challenging to estimate. But by assuming that the p covariates are independent within each class, we completely eliminate the need to worry about the association between the p predictors, because we have simply assumed that there is no association between the predictors!

Do we really believe the naive Bayes assumption that the p covariates are independent within each class? In most settings, we do not. But even though this modelling assumption is made for convenience, it often leads to pretty decent results, especially in settings where n is not large enough relative to p for us to effectively estimate the joint distribution of the predictors within each class. In fact, since estimating a joint distribution requires such a huge amount of data, naive Bayes is a good choice in a wide range of settings. Essentially, the naive Bayes assumption introduces some bias, but reduces variance, leading to a classifier that works quite well in practice as a result of the bias-variance trade-off.

A comparison of classification models

None of these methods uniformly dominates the others: in any setting, the choice of method will depend on the true distribution of the predictors in each of the K classes, as well as other considerations, such as the values of n and p. The latter ties into the bias-variance trade-off.

Chapter 5: Resampling methods

Resampling methods involve repeatedly drawing samples from a training set and refitting a model of interest on each sample in order to obtain additional information about the fitted model. Compared to only fitting a model once, we could attain more valuable information by drawing different samples from the training data. In this chapter, we are discussing two different approaches to resampling: cross-validation and the bootstrap.

Cross-validation

In the absence of a very large, designated test set that can be directly used to estimate the test error rate, a number of techniques can be used to estimate this quantity using the available training data. Some methods make a mathematical adjustment to the training

error rate in order to estimate the test error rate. In this section, we are rather considering a set of methods that estimate the test error rate by holding out a subset of the training observations from the fitting process and then applying the statistical learning method to those held out observations.

The validation set approach

This approach is very simple in its approach and involves randomly dividing the available set into two parts, a training set and a validation set (hold-out set). Then, the model is fit on the training set and used to predict the responses for observations in the hold-out set. The resulting validation set error rate – typically assessed using MSE in the case of a quantitative set error rate – provides an estimate of the test error rate.

Leave-One-Out Cross-Validation

LOOCV is closely related to the previously discussed validation set approach but attempts to address the drawbacks of this method. In this method we are also splitting the training data into two parts. However, instead of creating two subsets of comparable size, a single observation is used for the validation set, and the remaining observations make up the training set. The statistical learning method is fit on the n-1 training observations, and a prediction $\hat{y}_1$ is made for the excluded observation, using its value x_1. Since this one point was not used in the fitting process, we can use it to compute an estimated error. Then, this process is iterated for all of the n observations in the training set. Then, the estimated error is computed as the average error:

$$CV_{(n)} = \frac{1}{n}\sum_{i=1}^{n} MSE_i$$

Compared to the validation set approach, LOOCV has a couple of major advantages. Firstly, it has far less bias. Seeing that we use almost the entire training set (n-1) every time compared to around half of the set for the validation set approach, LOOCV tends not to overestimate the test error rate as much. Secondly, while the validation set method yields different results every time due to randomness in splits, performing LOOCV multiple times will always yield the same results as there is no randomness in the splits.

On the other hand, this approach can be an expensive one, being very time consuming if n is large. But, with least squares linear or polynomial regression, an amazing shortcut makes the cost of LOOCV the same as that of a single model fit! The following formula holds:

$$CV_{(n)} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_1}{1 - h_i}\right)^2$$

Where $\hat{y}_1$ is the ith fitted value from the original least squares fit, and h_i is the leverage. LOOCV is a very general method and can be used with any kind of modelling.

k-Fold Cross-Validation

An alternative to LOOCV is k-fold CV. This approach involves randomly dividing the set of observations into k groups, or folds, of approximately equal size. The first fold is treated as a validation set, and the method is fit on the remaining k − 1 folds. The mean squared error, MSE_1, is then computed on the observations in the held-out fold. This procedure is repeated k times; each time, a different group of observations is treated as a validation set. This

process results in k estimates of the test error, $MSE_1, MSE_2, \ldots, MSE_k$. The k-fold CV estimate is computed by averaging these values,

$$CV_{(k)} = \frac{1}{k}\sum_{i=1}^{k} MSE_i$$

It is not hard to see that LOOCV is a special case of k-fold CV where k is set equal to n. In practice, one typically performs this approach with k=5 or k=10. But, what is the advantage of this approach compared to k=n? The most obvious one is computational. If the training set is too large, fitting the learning procedure only 5 or 10 times may simply be much more feasible. We also may touch on some non-computational advantages later, related to bias-variance trade-off.

When we perform cross-validation, our goal might be to determine how well a given statistical learning procedure can be expected to perform on independent data; in this case, the actual estimate of the test MSE is of interest. But at other times we are interested only in the location of the minimum point in the estimated test MSE curve. This is because we might be performing cross-validation on several statistical learning methods, or on a single method using different levels of flexibility, in order to identify the method that results in the lowest test error. For this purpose, the location of the minimum point in the estimated test MSE curve is important, but the actual value of the estimated test MSE is not.

Bias-variance trade-off for k-Fold Cross-Validation

The validation set approach can lead to overestimates of the test error rate since the amount of data used to fit the statistical learning method is only half the size of the entire training set. Using this logic, it is not hard to see that LOOCV will give approximately unbiased estimates of the test error, since each training set contains $n - 1$ observations, which is almost as many as the number of observations in the full data set. And performing k-fold CV for, say, $k = 5$ or $k = 10$ will lead to an intermediate level of bias, since each training set contains approximately $(k - 1)n/k$ observations— fewer than in the LOOCV approach, but substantially more than in the validation set approach. Therefore, from the perspective of bias reduction, LOOCV is to be preferred to k-fold CV.

However, we know that bias is not the only source for concern in an estimating procedure; we must also consider the procedure's variance. It turns out that LOOCV has higher variance than does k-fold CV with k < n. Why is this the case? When we perform LOOCV, we are in effect averaging the outputs of n fitted models, each of which is trained on an almost identical set of observations; therefore, these outputs are highly (positively) correlated with each other. In contrast, when we perform k-fold CV with k < n, we are averaging the outputs of k fitted models that are somewhat less correlated with each other, since the overlap between the training sets in has higher variance than does the mean of many quantities that are not as highly correlated, the test error estimate resulting from LOOCV tends to have higher variance than does the test error estimate resulting from k-fold CV.

To summarize, there is a bias-variance trade-off associated with the choice of k in k-fold cross-validation. Typically, given these considerations, one performs k-fold cross-validation using $k = 5$ or $k = 10$, as these values have been shown empirically to yield test error rate estimates that suffer neither from excessively high bias nor from very high variance.

Cross-Validation on classification problems

So far, we have discussed CV in a regression setting when Y is quantitative. But CV can also be very useful when Y is qualitative. The difference lays in the error measurement. Instead of using MSE, we use the number of misclassifications to quantify errors.

The Bootstrap

The bootstrap is a widely applicable and extremely powerful statistical tool that can be used to quantify the uncertainty associated with a given estimator or statistical learning method. As a simple example, it can be used to estimate standard errors of the coefficients from a linear regression fit. As opposed to linear regression where these standard errors are easily attainable, its power lies in the fact that it can be easily applied to a wide range of statistical learning methods, including some for which a measure of variability is otherwise too difficult to obtain and is not automatically output by statistical software.

From wiki:

Bootstrapping is a procedure for estimating the distribution of an estimator by resampling (often with replacement) one's data or a model estimated from the data. Bootstrapping assigns measures of accuracy (bias, variance, confidence intervals, prediction error, etc.) to sample estimates. This technique allows estimation of the sampling distribution of almost any statistic using random sampling methods.

Bootstrapping estimates the properties of an estimand (such as its variance) by measuring those properties when sampling from an approximating distribution. One standard choice for an approximating distribution is the empirical distribution function of the observed data. In the case where a set of observations can be assumed to be from an independent and identically distributed population, this can be implemented by constructing a number of resamples with replacement, of the observed data set (and of equal size to the observed data set). A key result in Efron's seminal paper that introduced the bootstrap is the favorable performance of bootstrap methods using sampling with replacement compared to prior methods like the jackknife that sample without replacement. However, since its introduction, numerous variants on the bootstrap have been proposed, including methods that sample without replacement or that create bootstrap samples larger or smaller than the original data.

The bootstrap may also be used for constructing hypothesis tests. It is often used as an alternative to statistical inference based on the assumption of a parametric model when that assumption is in doubt, or where parametric inference is impossible or requires complicated formulas for the calculation of standard errors.

Chapter 6: Linear Model Selection and Regularization

Before moving on into the non-linear world, we want to explore how the simple linear model can be improved, as it often shows to compete well with more advanced models. We replace the least squares fitting process with other methods, but why? Alternative fitting processes can yield better prediction accuracy and model interpretability:

- Prediction accuracy: If n, the number of test observations, is not much larger than p, the number of predictors there can be a lot of variability in the least squares fit,

resulting in overfitting. And if p is larger than n, there is no longer a unique least squares coefficient estimate: there are infinitely many solutions. Thus, by shrinking or constraining the estimated coefficients, we can substantially reduce the variance at the cost of a negligible increase in bias, leading to better predictive accuracy

- Model interpretability: Irrelevant variables lead to unnecessary complexity in our models. By removing these variables - that is, by setting the corresponding coefficients to zero – we can obtain a model that is more easily interpreted.

In this chapter we will look at methods for variable selection or feature selection – excluding irrelevant variables from a multiple regression model. We discuss three important classes of methods:

- Subset selection: Identifying a subset of the p predictors that we believe to be related to the response. Then fit a least squares model with these
- Shrinkage: Fitting a model involving all p predictors, but the estimated coefficients are shrunken towards zero relative to the least squares estimates. This is also called regularization and has the effect of reducing variance. As some of these methods shrink coefficients down to zero, they can also perform variable selection
- Dimension reduction: This involves projecting the p predictors into an M-dimensional subspace, where M < p. This is achieved by computing M different linear combinations, or projections, of the variables. Then these M projections are used as predictors to fit a linear regression model by least squares.

Subset selection

Best subset selection

For this type of subset selection we fit a separate least squares regression for each possible combination of the p predictors. That is, we fit all p models that contain exactly one predictors, all models that contain exactly two predictors and so forth. We then look to the one that is the best.

Algorithm 6.1 *Best subset selection*

1. Let $\mathcal{M}_0$ denote the *null model*, which contains no predictors. This model simply predicts the sample mean for each observation.
2. For $k = 1, 2, \ldots p$:
   (a) Fit all $\binom{p}{k}$ models that contain exactly k predictors.
   (b) Pick the best among these $\binom{p}{k}$ models, and call it $\mathcal{M}_k$. Here *best* is defined as having the smallest RSS, or equivalently largest R^2.
3. Select a single best model from among $\mathcal{M}_0, \ldots, \mathcal{M}_p$ using using the prediction error on a validation set, C_p (AIC), BIC, or adjusted R^2. Or use the cross-validation method.

While best subset selection is a simple and conceptually appealing approach, it suffers from computational limitations. The number of possible models that must be considered grows rapidly as p increases. In general, there are 2^p models that involve subsets of p predictors. So, if p = 10, then there are approximately 1,000 possible models to be considered, and if p =

20, then there are over one million possibilities! Consequently, best subset selection becomes computationally infeasible for values of p greater than around 40, even with extremely fast modern computers. There are computational shortcuts—so called branch-and-bound techniques—for eliminating some choices, but these have their limitations as p gets large. They also only work for least squares linear regression.

Stepwise selection

Stepwise selection explores a far more restricted set of models and are attractive alternatives to best subset selection.

Forward stepwise selection

Forward stepwise selection begins with a model containing no predictors, and then adds predictors to the model, one-at-a-time, until all of the predictors are in the model. In particular, at each step the variable that gives the greatest additional improvement to the fit is added to the model. More formally, the forward stepwise selection procedure is given below:

Algorithm 6.2 *Forward stepwise selection*

1. Let $\mathcal{M}_0$ denote the *null* model, which contains no predictors.
2. For $k = 0, \ldots, p-1$:
   (a) Consider all $p-k$ models that augment the predictors in $\mathcal{M}_k$ with one additional predictor.
   (b) Choose the *best* among these $p-k$ models, and call it $\mathcal{M}_{k+1}$. Here *best* is defined as having smallest RSS or highest R^2.
3. Select a single best model from among $\mathcal{M}_0, \ldots, \mathcal{M}_p$ using the prediction error on a validation set, C_p (AIC), BIC, or adjusted R^2. Or use the cross-validation method.

Though forward stepwise tends to do well in practice, it is not guaranteed to find the best possible model out of all 2p models containing subsets of the p predictors. For instance, suppose that in a given data set with p = 3 predictors, the best possible one-variable model contains X1, and the best possible two-variable model instead contains X2 and X3. Then forward stepwise selection will fail to select the best possible two-variable model, because M1 will contain X1, so M2 must also contain X1 together with one additional variable.

Backward stepwise selection

Unlike forward stepwise selection, backward selection begins with the full least squares model containing all p predictors, and then iteratively removes the least useful predictor, one-at a- time. Details are given below

Algorithm 6.3 *Backward stepwise selection*

1. Let $\mathcal{M}_p$ denote the *full* model, which contains all p predictors.
2. For $k = p, p-1, \ldots, 1$:
   (a) Consider all k models that contain all but one of the predictors in $\mathcal{M}_k$, for a total of $k-1$ predictors.
   (b) Choose the *best* among these k models, and call it $\mathcal{M}_{k-1}$. Here *best* is defined as having smallest RSS or highest R^2.
3. Select a single best model from among $\mathcal{M}_0, \ldots, \mathcal{M}_p$ using the prediction error on a validation set, C_p (AIC), BIC, or adjusted R^2. Or use the cross-validation method.

Like forward stepwise selection, the backward selection approach searches through only $1 + p(p+1)/2$ models and so can be applied in settings where p is too large to apply best subset selection. Also, like forward stepwise selection, backward stepwise selection is not guaranteed to yield the best model containing a subset of the p predictors. Backward selection requires that the number of samples n is larger than the number of variables p (so that the full model can be fit). In contrast, forward stepwise can be used even when n < p, and so is the only viable subset method when p is very large.

Hybrid approaches

The best subset, forward stepwise, and backward stepwise selection approaches generally give similar but not identical models. As another alternative, hybrid versions of forward and backward stepwise selection are available, in which variables are added to the model sequentially, in analogy to forward selection. However, after adding each new variable, the method may also remove any variables that no longer provide an improvement in the model fit. Such an approach attempts to more closely mimic best subset selection while retaining the computational advantages of forward and backward stepwise selection.

Choosing the optimal model

Best subset selection, forward selection, and backward selection result in the creation of a set of models, each of which contains a subset of the p predictors. To apply these methods, we need a way to determine which of these models is best. As we discussed in Section 6.1.1, the model containing all the predictors will always have the smallest RSS and the largest R^2, since these quantities are related to the training error. Instead, we wish to choose a model with a low test error. As is evident here, and as we show in Chapter 2, the training error can be a poor estimate of the test error. Therefore, RSS and R^2 are not suitable for selecting the best model among a collection of models with different numbers of predictors. In order to select the best model with respect to test error, we need to estimate this test error. There are two common approaches:

1. We can indirectly estimate test error by making an adjustment to the training error to account for the bias due to overfitting
2. We can directly estimate the test error, using either a validation set approach or a cross-validation approach, as discussed in chapter 5

C_p, AIC, BIC and adjusted R^2

Criterion	Formula	Penalizes Model Complexity?	How It Adjusts Training Error	Preferred Value
Mallows' C_p	$C_p = (1/n) \times [RSS + 2d\sigma^2]$	Yes ($2d\sigma^2$ term)	Adds penalty for number of predictors	Close to d
AIC	$AIC = n \times \log(RSS/n) + 2d$	Yes (2d term)	Penalizes log-likelihood by number of predictors	Lower is better
BIC	$BIC = n \times \log(RSS/n) + d \times \log(n)$	Yes ($d \times \log(n)$ term)	Harsher penalty as n increases	Lower is better
Adjusted R^2	Adjusted $R^2 = 1 - [(RSS/(n-d-1)) / (TSS/(n-1))]$	Yes (by degrees of freedom)	Adjusts R^2 downward for number of predictors	Higher is better

Validation and cross-validation

mate the test error using the validation set and cross-validation methods discussed in Chapter 5. We can compute the validation set error or the cross-validation error for each model under consideration, and then select the model for which the resulting estimated test error is smallest. This procedure has an advantage relative to AIC, BIC, Cp, and adjusted R2, in that it provides a direct estimate of the test error and makes fewer assumptions about the true underlying model. It can also be used in a wider range of model selection tasks, even in cases where it is hard to pinpoint the model degrees of freedom (e.g. the number of predictors in the model) or hard to estimate the error variance σ^2.

Shrinkage methods

The subset selection methods described in Section 6.1 involve using least squares to fit a linear model that contains a subset of the predictors. As an alternative, we can fit a model containing all p predictors using a technique that constrains or regularizes the coefficient estimates, or equivalently, that shrinks the coefficient estimates towards zero. It may not be immediately obvious why such a constraint should improve the fit, but it turns out that shrinking the coefficient estimates can significantly reduce their variance. The two best-known techniques for shrinking the regression coefficients towards zero are ridge regression and the lasso.

Ridge regression

Ridge regression is very similar to least squares, except that the coefficients are estimated by minimizing a slightly different quantity. In particular, the ridge regression coefficient estimates $\hat{\beta}^R$ are the values that minimize

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}\beta_j^2 = RSS + \lambda\sum_{j=1}^{p}\beta_j^2$$

Where $\lambda \geq 0$ is a tuning parameter, to be determined separately. The equation above trades off two different criteria. As with least squares, ridge regression seeks coefficient estimates that fit the data well, by making the RSS small. However, the second term, $\lambda\sum_{j=1}^{p}\beta_j^2$, called a shrinkage penalty, is small when $\beta_1, \ldots, \beta_p$ are close to zero, and so it has the effect of shrinking the estimates of β_j towards zero. The tuning parameter λ serves to control the relative impact of these two terms on the regression coefficient estimates. When λ = 0, the penalty term has no effect, and ridge regression will produce the least squares estimates. However, as λ→∞, the impact of the shrinkage penalty grows, and the ridge regression coefficient estimates will approach zero. Unlike least squares, which generates only one set of coefficient estimates, ridge regression will produce a different set of coefficient estimates, $\hat{\beta}_\lambda^R$, for each value of λ. Selecting a good value for λ is critical; we defer this discussion to Section 6.2.3, where we use cross-validation.

The lasso

Ridge regression does have one obvious disadvantage. Unlike best subset, forward stepwise, and backward stepwise selection, which will generally select models that involve just a subset of the variables, ridge regression will include all p predictors in the final model. The penalty $\lambda\sum_{j=1}^{p}\beta_j^2$ in (6.5) will shrink all of the coefficients towards zero, but it will not set

any of them exactly to zero (unless λ = ∞). This may not be a problem for prediction accuracy, but it can create a challenge in model interpretation in settings in which the number of variables p is quite large. The lasso is a relatively recent alternative to ridge regression that overcomes this disadvantage. The lasso coefficients, $\hat{\beta}_{\lambda}^{L}$, minimize the quantity

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}|\beta_j| = RSS + \lambda\sum_{j=1}^{p}|\beta_j|$$

Comparing this formula to the ridge, we see clear similarities. As with ridge regression, the lasso shrinks the coefficient estimates towards zero. However, in the case of the lasso, the ℓ1 penalty has the effect of forcing some of the coefficient estimates to be exactly equal to zero when the tuning parameter λ is sufficiently large. Hence, much like best subset selection, the lasso performs variable selection. As a result, models generated from the lasso are generally much easier to interpret than those produced by ridge regression. We say that the lasso yields sparse models—that is, sparse models that involve only a subset of the variables. As in ridge regression, selecting a good value of λ for the lasso is critical; we defer this discussion to Section 6.2.3, where we use cross-validation.

Dimension reduction methods

The methods that we have discussed so far in this chapter have controlled variance in two different ways, either by using a subset of the original variables, or by shrinking their coefficients toward zero. All these methods are defined using the original predictors, $X_1, X_2, \ldots, X_p$. We now explore a class of approaches that transform the predictors and then fit a least squares model using the transformed variables. We will refer to these techniques as dimension reduction methods.

Let Z_1, Z_2, …, Z_M represent M < p linear combinations of our original p predictors. That is,

$$Z_M = \sum_{j=1}^{p}\phi_{jm}X_j$$

For some constants ϕ_{1m}, ϕ_{2m}, …, ϕ_{pm}, m = 1, …, M. We can then fit the linear regression model

$$y_i = \theta_0 + \sum_{m=1}^{M}\theta_m z_{im} + \epsilon_i \qquad i = 1, \ldots, n$$

Using least squares. Note that in equation above, the regression coefficients are given by $\theta_0, \theta_1, \ldots, \theta_M$. If the constants $\varphi_{1m}, \varphi_{2m}, \ldots, \varphi_{pm}$ are chosen wisely, then such dimension reduction approaches can often outperform least squares regression. In other words, fitting using least squares can lead to better results than fitting (6.1) using least squares. The term dimension reduction comes from the fact that this approach reduces the problem of estimating the $p + 1$ coefficients $\beta_0, \beta_1, \ldots, \beta_p$ to the simpler problem of estimating the $M + 1$ coefficients $\theta_0, \theta_1, \ldots, \theta_M$, where $M < p$. In other words, the dimension of the problem has been reduced from $p + 1$ to $M + 1$.

All dimension reduction methods work in two steps. First, the transformed predictors $Z_1, Z_2, \ldots, Z_M$ are obtained. Second, the model is fit using these M predictors. However, the

choice of $Z_1, Z_2, \ldots, Z_M$, or equivalently, the selection of the φ_{jm}'s, can be achieved in different ways. In this chapter, we will consider two approaches for this task: principal components and partial least squares.

Principal components regression

Principal components analysis (PCA) is a popular approach for deriving a low-dimensional set of features from a large set of variables. PCA is discussed in greater detail as a tool for unsupervised learning in Chapter 12. Here we describe its use as a dimension reduction technique for regression.

The principal components regression (PCR) approach involves constructing the first M principal components, $Z_1, \ldots, Z_M$, and then using these components as the predictors in a linear regression model that is fit using least squares. The key idea is that often a small number of principal components suffice to explain most of the variability in the data, as well as the relationship with the response. In other words, we assume that the directions in which $X_1, \ldots, X_p$ show the most variation are the directions that are associated with Y. While this assumption is not guaranteed to be true, it often turns out to be a reasonable enough approximation to give good results.

If the assumption underlying PCR holds, then fitting a least squares model to $Z_1, \ldots, Z_M$ will lead to better results than fitting a least squares model to $X_1, \ldots, X_p$, since most or all of the information in the data that relates to the response is contained in $Z_1, \ldots, Z_M$, and by estimating only $M \ll p$ coefficients we can mitigate overfitting.

Chapter 7: Moving Beyond Linearity

Standard linear regressions can have strong limitations when it comes to predictive power. This is because the linearity assumption is almost always an approximation, and sometimes a poor one. In this chapter we will relax the linearity assumption while still attempting to maintain as much interpretability as possible.

Polynomial regression

This has historically been the standard way of extending the linear regression to accommodate for non-linear relationships, replacing the standard regression with a polynomial regression:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \cdots + \beta_d x_i^d + \epsilon_i$$

For a large enough degree of d, we can produce an extremely non-linear curve. Still, this model can be easily estimated with least squares. Generally speaking, it is unusual to use d greater than 3 or 4 because for large values of d, the polynomial curve can become overly flexible and can take on some very strange shapes.

Step functions

Using polynomial functions of the features as predictors in a linear model imposes a global structure on the non-linear function of X. We can instead use step functions in order to avoid imposing such a global structure. Here we break the range of X into bins, and fit a different constant in each bin. This amounts to converting a continuous variable into an ordered

categorical variable. In greater detail, we create cut points $c_1, c_2, \ldots, c_K$ in the range of X, and then construct $K + 1$ new variables

$$\begin{aligned} C_0(X) &= I(X < c_1), \\ C_1(X) &= I(c_1 \le X < c_2), \\ C_2(X) &= I(c_2 \le X < c_3), \\ &\vdots \\ C_{K-1}(X) &= I(c_{K-1} \le X < c_K), \\ C_K(X) &= I(c_K \le X), \end{aligned} \tag{7.4}$$

Where I(·) is an indicator function that returns 1 if the condition is true, and returns 0 otherwise. The functions work as dummies so that for a given value of X, at most one of C_1, $C_2, \ldots, C_K$ can be non-zero.

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \cdots + \beta_K C_K(x_i) + \epsilon_i$$

Unfortunately, unless there are natural breakpoints in the predictors, piecewise-constant functions can miss the action. Nevertheless, step function approaches are very popular in biostatistics and epidemiology, among other disciplines. For example, 5-year age groups are often used to define the bins.

Basis functions

Polynomial and piecewise-constant regression models are in fact special cases of a basis function approach. The idea is to have a family of functions or transformations that can be applied to variable X: $b_1(X), b_2(X), \ldots, b_k(X)$. Instead of fitting a linear model in X, we fit the model

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_K b_K(x_i) + \epsilon_i$$

We note that the basis functions $b_1(\cdot), b_2(\cdot), \ldots, b_k(\cdot)$ are fixed and known. (In other words, we choose the functions ahead of time.) We can still think of it as a standard linear model with predictors, hence using least squares to estimate unknown coefficients.

Regression splines

Now we discuss a flexible class of basis functions that extends upon the polynomial regression and piecewise constant regression approaches that we have just seen.

Piecewise polynomials

Instead of fitting a high-degree polynomial over the entire range of X, piecewise polynomial regression involves fitting separate low-degree polynomials over different regions of X. For example, a piecewise cubic polynomial works by fitting a cubic regression model of the form

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \epsilon_i$$

where the coefficients β_0, β_1, β_2, and β_3 differ in different parts of the range of X. The points where the coefficients change are called knots. For example, a piecewise cubic with no knots is just a standard cubic polynomial, as in (7.1) with d = 3. A piecewise cubic polynomial with a single knot at a point c takes the form

$$y_i = \begin{cases} \beta_{01} + \beta_{11} x_i + \beta_{21} x_i^2 + \beta_{31} x_i^2 + \epsilon_i & if\ x_i < c \\ \beta_{02} + \beta_{12} x_i + \beta_{22} x_i^2 + \beta_{32} x_i^2 + \epsilon_i & if\ x_i \ge c \end{cases}$$

In other words, we fit two different polynomial functions to the data, one on the subset of the observations with $x_i < c$, and one on the subset of the observations with $x_i \geq c$.

Using more knots leads to a more flexible piecewise polynomial. In general, if we place K different knots throughout the range of X, then we will end up fitting K +1 different cubic polynomials. Note that we do not need to use a cubic polynomial. For example, we can instead fit piecewise linear functions. In fact, our piecewise constant functions of Section 7.2 are piecewise polynomials of degree 0!

Choosing the number and locations of the knots

When we fit a spline, where should we place the knots? The regression spline is most flexible in regions that contain a lot of knots, because in those regions the polynomial coefficients can change rapidly. Hence, one option is to place more knots in places where we feel the function might vary more rapidly, and to place fewer knots where it seems more stable. While this option can work well, in practice it is common to place knots in a uniform fashion. One way to do this is to specify the desired degrees of freedom, and then have the software automatically place the corresponding number of knots at uniform quantiles of the data.

How many knots should we use, or equivalently how many degrees of freedom should our spline contain? One option is to try out different numbers of knots and see which produces the best-looking curve. A somewhat more objective approach is to use cross-validation, as discussed in Chapters 5 and 6. With this method, we remove a portion of the data (say 10 %), fit a spline with a certain number of knots to the remaining data, and then use the spline to make predictions for the held-out portion. We repeat this process multiple times until each observation has been left out once and then compute the overall cross-validated RSS. This procedure can be repeated for different numbers of knots K. Then the value of K giving the smallest RSS is chosen.

Comparison to polynomial regression

Regression splines often give superior results to polynomial regression. This is because unlike polynomials, which must use a high degree (exponent in the highest monomial term, e.g. X^{15}) to produce flexible fits, splines introduce flexibility by increasing the number of knots but keeping the degree fixed. Generally, this approach produces more stable estimates. Splines also allow us to place more knots, and hence flexibility, over regions where the function f seems to be changing rapidly, and fewer knots where f appears more stable.

Smoothing splines

We now introduce a somewhat different approach that also produces a spline. In fitting a smooth curve to a set of data, what we really want to do is find some function, say g(x), that fits the observed data well: that is, we want RSS to be small. However, there is a problem with this approach. If we don't put any constraints on $g(x_i)$, then we can always make RSS zero simply by choosing g such that it interpolates all of the y_i. Such a function would woefully overfit the data—it would be far too flexible. What we really want is a function g that makes RSS small, but that is also smooth. How might we ensure that g is smooth? There are a number of ways to do this. A natural approach is to find the function g that minimizes

$$\sum_{i=1}^{n} \left(y_i - g(x_i)\right)^2 + \lambda \int g''(t)^2 dt$$

Where λ is a non-negative tuning parameter. The function g that minimizes the function above is known as a smoothing spline. But what does this function actually mean? The term $\sum_{i=1}^{n}\left(y_i - g(x_i)\right)^2$ is a loss function that encourages g to fit the data well, and the term $\lambda \int g''(t)^2 dt$ is a penalty term the penalizes the variability in g. The notation $g''(t)$ indicates the second derivative of the function g. The first derivative measures the slope, and the second derivative measures the rate at which the slope is changing. Hence, broadly speaking, the second derivative of a function is a measure of its roughness. In other words, $\int g''(t)^2 dt$ is simply a measure of the total change in the function $g'(t)$, over its entire range. If g is very smooth, then $g'(t)$ will be close to constant and $\int g''(t)^2 dt$ will take on a small value. Conversely, if g is jumpy and variable then g′(t) will vary significantly and $\int g''(t)^2 dt$ will take on a large value. Therefore, in (7.11), $\lambda \int g''(t)^2 dt$ encourages g to be smooth. The larger the value of λ, the smoother g will be.

Choosing the smoothing parameter λ

We have seen that a smoothing spline is simply a natural cubic spline with knots at every unique value of x_i. It might seem that a smoothing spline will have far too many degrees of freedom, since a knot at each data point allows a great deal of flexibility. But the tuning parameter λ controls the roughness of the smoothing spline, and hence the effective degrees of freedom. It is possible to show that as λ increases from 0 to ∞, the effective effective degrees of freedom degrees of freedom, which we write df_λ, decrease from n to 2.

Although a smoothing spline has n parameters and hence n nominal degrees of freedom, these n parameters are heavily constrained or shrunk down. Hence df_λ is a measure of the flexibility of the smoothing spline—the higher it is, the more flexible (and the lower-bias but higher-variance) the smoothing spline. In fitting a smoothing spline, we do not need to select the number or location of the knots—there will be a knot at each training observation, $x_1, \ldots, x_n$. Instead, we have another problem: we need to choose the value of λ. It should come as no surprise that one possible solution to this problem is cross-validation. In other words, we can find the value of λ that makes the cross-validated RSS as small as possible. It turns out that the leave-one-out cross-validation error (LOOCV) can be computed very efficiently for smoothing splines, with essentially the same cost as computing a single fit.

Local regression

Local regression is a different approach for fitting flexible, non-linear functions, which involves computing the fit at a target point x_0 using only the nearby training observations. Local regression is sometimes referred to as a memory-based procedure, because like nearest-neighbors, we need all the training data each time we wish to compute a prediction.

In order to perform local regression, there are a number of choices to be made, such as how to define the weighting function K, and whether to fit a linear, constant, or quadratic regression in Step 3. (Equation 7.14 corresponds to a linear regression.) While all of these choices make some difference, the most important choice is the span s, which is the proportion of points used to compute the local regression at x0, as defined in Step 1 above. The span plays a role like that of the tuning parameter λ in smoothing splines: it controls the flexibility of the non-linear fit. The smaller the value of s, the more local and wiggly will be our fit; alternatively, a very large value of s will lead to a global fit to the data using all of the training observations. The algorithm looks like so:

Algorithm 7.1 *Local Regression At $X = x_0$*

1. Gather the fraction $s = k/n$ of training points whose x_i are closest to x_0.
2. Assign a weight $K_{i0} = K(x_i, x_0)$ to each point in this neighborhood, so that the point furthest from x_0 has weight zero, and the closest has the highest weight. All but these k nearest neighbors get weight zero.
3. Fit a *weighted least squares regression* of the y_i on the x_i using the aforementioned weights, by finding $\hat{\beta}_0$ and $\hat{\beta}_1$ that minimize

$$\sum_{i=1}^{n} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2. \tag{7.14}$$

4. The fitted value at x_0 is given by $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.

Again, we can use cross-validation to choose s, or we can specify it directly. The idea of local regression can be generalized in many different ways. In a setting with multiple features $X_1, X_2, \ldots, X_p$, one very useful generalization involves fitting a multiple linear regression model that is global in some variables, but local in another, such as time. Such varying coefficient models are a useful way of adapting a model to the most recently gathered varying coefficient model data. Local regression also generalizes very naturally when we want to fit models that are local in a pair of variables X_1 and X_2, rather than one. We can simply use two-dimensional neighborhoods, and fit bivariate linear regression models using the observations that are near each target point in two-dimensional space. Theoretically the same approach can be implemented in higher dimensions, using linear regressions fit to p-dimensional neighborhoods. However, local regression can perform poorly if p is much larger than about 3 or 4 because there will generally be very few training observations close to x_0.

Generalized Additive Models

Here, we explore the problem of flexibly predicting Y based on several predictors, $X_1, \ldots, X_p$. This amounts to an extension of multiple linear regression. Generalized additive models (GAMs) provide a general framework for extending a standard linear model by allowing non-linear functions of each of the variables, while maintaining additivity. Just like linear models, GAMs can be applied with both quantitative and qualitative responses.

GAMs for regression problems

A natural way to extend the multiple linear regression model in order to allow for non-linear relationships between each feature and the response is to replace each linear component $\beta_j x_{ij}$ with a (smooth) non-linear function $f(x_{ij})$. We would then write the model as

$$\begin{aligned} y_i &= \beta_0 + \sum_{j=1}^{p} f_j(x_{ij}) + \epsilon_i \\ &= \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \epsilon_i. \end{aligned} \tag{7.15}$$

This is an example of a GAM. It is called an additive model because we calculate a separate f_j for each X_j, and then add together all of their contributions. The beauty of GAMs is that

we can use the methods mentioned previously in the chapter as building blocks for fitting an additive model. In fact, for most of the methods that we have seen so far in this chapter, this can be done fairly trivially.

Fitting a GAM with a smoothing spline is not quite as simple as fitting a GAM with a natural spline, since in the case of smoothing splines, least squares cannot be used. However, standard software such as the gam() function in R can be used to fit GAMs using smoothing splines, via an approach known as backfitting. This method fits a model involving multiple predictors by repeatedly updating the fit for each predictor in turn, holding the others backfitting fixed. The beauty of this approach is that each time we update a function, we simply apply the fitting method for that variable to a partial residual.

We do not have to use splines as the building blocks for GAMs: we can just as well use local regression, polynomial regression, or any combination of the approaches seen earlier in this chapter in order to create a GAM. GAMs are investigated in further detail in the lab at the end of this chapter.

Pros and cons of GAMs

Pros of GAMs	Cons of GAMs
Can automatically fit non-linear relationships for each predictor, avoiding manual transformation trials	Restricted to additive effects—cannot model complex interactions by default
Capture non-linear effects standard linear regression would miss	May miss important variable interactions if not explicitly included
Can improve prediction accuracy over linear models due to non-linearity	Interaction effects only included if manually specified via terms like $X_j \times X_k$ or two-dimensional smoothers
Additive structure allows interpretation of each predictor's effect, holding others fixed	For very flexible modeling of interactions, fully nonparametric methods (e.g., random forests, boosting) may be better
The smoothness of each function can be summarized and interpreted via degrees of freedom	

The same application of GAM models applies to qualitative response variables.

Chapter 8: Tree-Based Methods

In this chapter, we describe tree-based methods for regression and classification. These involve stratifying or segmenting the predictor space into a number of simple regions. In order to make a prediction for a given observation, we typically use the mean or the mode response value for the training observations in the region to which it belongs. Since the set of splitting rules used to segment the predictor space can be summarized in a tree, these types of approaches are known as decision tree methods.

Regression Trees

At each step, the tree selects the feature and threshold that maximizes the reduction in prediction error, typically measured by Mean Squared Error (MSE) or RSS. The splitting process continues iteratively for each new node until a stopping criterion is met, such as reaching a maximum tree depth, a minimum number of observations per leaf, or a lack of significant error reduction. Once the tree is constructed, predictions for new observations are obtained by traversing the tree based on feature values until a terminal (leaf) node is reached. The predicted value is then the average of the training observations within that leaf.

Tree pruning

The process described above may produce good predictions on the training set, but is likely to overfit the data, leading to poor test set performance. This is because the resulting tree might be too complex. A smaller tree with fewer splits (that is, fewer regions $R_1, \ldots, R_J$) might lead to lower variance and better interpretation at the cost of a little bias. One possible alternative to the process described above is to build the tree only so long as the decrease in the RSS due to each split exceeding some (high) threshold. This strategy will result in smaller trees but is too short-sighted since a seemingly worthless split early on in the tree might be followed by a very good split—that is, a split that leads to a large reduction in RSS later on.

Therefore, a better strategy is to grow a very large tree T_0, and then prune it back in order to obtain a subtree. How do we determine the best way to prune the tree? Intuitively, our goal is to select a subtree that leads to the lowest test error rate. Given a subtree, we can estimate its test error using cross-validation or the validation set approach. However, estimating the cross-validation error for every possible subtree would be too cumbersome, since there is an extremely large number of possible subtrees. Instead, we need a way to select a small set of subtrees for consideration.

Cost complexity pruning, also known as weakest link pruning, gives us a way to do just this. Rather than considering every possible subtree, we consider a sequence of trees indexed by a nonnegative tuning parameter α. For each value of α there corresponds a subtree T ⊂ T0 such that

$$\sum_{m=1}^{|T|} \sum_{i:\, x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha |T|$$

Algorithm 8.1 *Building a Regression Tree*

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α.
3. Use K-fold cross-validation to choose α. That is, divide the training observations into K folds. For each $k = 1, \ldots, K$:

   (a) Repeat Steps 1 and 2 on all but the kth fold of the training data.

   (b) Evaluate the mean squared prediction error on the data in the left-out kth fold, as a function of α.

   Average the results for each value of α, and pick α to minimize the average error.
4. Return the subtree from Step 2 that corresponds to the chosen value of α.

is as small as possible. Here |T| indicates the number of terminal nodes of the tree T, R_m is the rectangle (i.e. the subset of predictor space) corresponding to the mth terminal node, and $\hat{y}_{R_m}$ is the predicted response associated with R_m—that is, the mean of the training observations in R_m. The tuning parameter α controls a trade-off between the subtree's complexity and its fit to the training data. When α = 0, then the subtree T will simply equal T_0, because then (8.4) just measures the training error. However, as α increases, there is a price to pay for having a tree with many terminal nodes, and so the quantity (8.4) will tend to be minimized for a smaller subtree.

Classification trees

A classification tree is very similar to a regression tree, except that it is used to predict a qualitative response rather than a quantitative one. Recall that for a regression tree, the predicted response for an observation is given by the mean response of the training observations that belong to the same terminal node. In contrast, for a classification tree, we predict that each observation belongs to the most commonly occurring class of training observations in the region to which it belongs. In interpreting the results of a classification tree, we are often interested not only in the class prediction corresponding to a particular terminal node region, but also in the class proportions among the training observations that fall into that region.

Since we plan to assign an observation in a given region to the most commonly occurring class of training observations in that region, the classification error rate is simply the fraction of the training observations in that region that do not belong to the most common class:

$$E = 1 - \max_k(\hat{p}_{mk})$$

Here, $\hat{p}_{mk}$ represents the proportion of training observations in the mth region that are from the kth class. However, it turns out that classification error is not sufficiently sensitive for tree-growing, and in practice two other measures are preferable. The Gini index is defined by:

$$G = \sum_{k=1}^{K} \hat{p}_{mk} log \hat{p}_{mk}$$

This is a measure of total variance across the K classes. It is not hard to see that the Gini index takes on a small value if all of the $\hat{p}_{mk}$'s are close to zero or one. For this reason, the Gini index is referred to as a measure of node purity, a small value indicates that a node contains predominantly observations from a single class. An alternative to the Gini index is entropy, given by

$$D = -\sum_{k=1}^{K} \hat{p}_{mk} log \hat{p}_{mk}$$

Since $0 \leq \hat{p}_{mk} \leq 1$, it follows that $0 \leq \hat{p}_{mk} log \hat{p}_{mk}$. One can show that the entropy will take on a value near zero if the $\hat{p}_{mk}$'s are all near zero or near one. Therefore, like the Gini index, the entropy will take on a small value if the mth node is pure. In fact, it turns out that the Gini index and the entropy are quite similar numerically. When building a classification tree, either the Gini index or the entropy are typically used to evaluate the quality of a particular split, since these two approaches are more sensitive to node purity than is the classification error rate. Any of these three approaches might be used when pruning the tree, but the classification error rate is preferable if prediction accuracy of the final pruned tree is the goal.

Advantages and disadvantages of trees

Advantages of Decision Trees	Disadvantages of Decision Trees
Very easy to explain and interpret, even easier than linear regression	Generally have lower predictive accuracy compared to other regression and classification approaches
Mirror human decision-making processes more closely than traditional statistical methods	Can be non-robust: small changes in the data can lead to very different trees
Easily visualized graphically, making results accessible to non-experts (especially if trees are small)	
Can naturally handle qualitative (categorical) predictors without needing to create dummy variables	
	Aggregating many trees (bagging, random forests, boosting) can improve predictive performance and stability

Bagging, Random Forests, Boosting, and Bayesian Additive Regression Trees

An ensemble method is an approach that combines many simple "building block" models in order to obtain a single and potentially very powerful model. These simple building block models are sometimes known as weak learners, since they may lead to mediocre predictions on their own. We will now discuss bagging, random forests, boosting, and Bayesian additive regression trees. These are ensemble methods for which the simple building block is a regression or a classification tree.

Bagging

The bootstrap, introduced in chapter 5, is an extremely powerful idea. It is used in many situations in which it is hard or even impossible to directly compute the standard deviation of a quantity of interest. We see here that this tool can be used in a completely different context in order to improve statistical learning methods such as decision trees.

Bootstrap aggregation, or bagging, is a general-purpose procedure for reducing the variance of a statistical learning method. We recall that a set of n different observations $Z_1, \dots, Z_n$ each with variance σ^2, the variance of the mean Z of the observations is given by $\frac{\sigma^2}{n}$. In other words, averaging a set of observations reduces variance. Hence, a natural way to reduce the variance and increase the test set accuracy is to take many training sets from the population, build a separate prediction model using each training set, and average the resulting predictions. In other words, we could calculate $\hat{f}^1(x), \hat{f}^2(x), \dots, \hat{f}^B(x)$ using B separate training sets, and average them in order to obtain a single low-variance statistical learning model, given by

$$\hat{f}_{avg}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^B(x)$$

Of course, this is not practical because we generally do not have access to multiple training sets. Instead, we can bootstrap by taking repeated samples from the (single) training data set. In this approach we generate B different bootstrapped training data sets. We then train our method on the bth bootstrapped training set in order to get $\hat{f}^B(x)$, and finally average all the predictions, to obtain

$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*B}(x)$$

This is called bagging. While bagging can improve predictions for many regression methods, it is particularly useful for decision trees. To apply bagging to regression trees, we simply construct B regression trees using B bootstrapped training sets and average the resulting predictions. These trees are grown deep and are not pruned. Hence each individual tree has a high variance, but low bias. Averaging these B trees reduces the variance. Bagging has been demonstrated to give impressive improvements in accuracy by combining hundreds or even thousands of trees into a single procedure.

How can bagging be extended to a classification problem where Y is qualitative? In that situation, there are a few possible approaches, but the simplest is as follows. For a given test observation, we can record the class predicted by each of the B trees and take a majority vote: the overall prediction is the most commonly occurring class among the B predictions.

Out-of-bag error estimation

It turns out that there is a very straightforward way to estimate the test error of a bagged model, without the need to perform cross-validation or the validation set approach. Recall that the key to bagging is that trees are repeatedly fit to bootstrapped subsets of the observations. One can show that on average, each bagged tree makes use of around two-thirds of the observations. The remaining one-third of the observations not used to fit a given bagged tree are referred to as the out-of-bag (OOB) observations. We can predict the response for the ith observation using each of the trees in which that observation was OOB.

This will yield around B/3 predictions for the ith observation. In order to obtain a single prediction for the ith observation, we can average these predicted responses (if regression is the goal) or can take a majority vote (if classification is the goal). This leads to a single OOB prediction for the ith observation. An OOB prediction can be obtained in this way for each of the n observations, from which the overall OOB MSE (for a regression problem) or classification error (for a classification problem) can be computed. The resulting OOB error is a valid estimate of the test error for the bagged model, since the response for each observation is predicted using only the trees that were not fit using that observation.

Variable importance measures

Although the collection of bagged trees is much more difficult to interpret than a single tree, one can obtain an overall summary of the importance of each predictor using the RSS (for bagging regression trees) or the Gini index (for bagging classification trees). In the case of bagging regression trees, we can record the total amount that the RSS (8.1) decreases due to splits over a given predictor, averaged over all B trees. A large value indicates an important predictor. Similarly, in the context of bagging classification trees, we can add up the total amount that the Gini index (8.6) is decreased by splits over a given predictor, averaged over all B trees.

Random forests

Random forests provide an improvement over bagged trees by way of a random small tweak that decorrelates the trees. As in bagging, we build a number forest of decision trees on bootstrapped training samples. But when building these decision trees, each time a split in a tree is considered, a random sample of m predictors is chosen as split candidates from the full set of p predictors. The split is allowed to use only one of those m predictors. A fresh sample of m predictors is taken at each split, and typically we choose $m \approx \sqrt{p}$ that is, the number of predictors considered at each split is approximately equal to the square root of the total number of predictors.

In other words, in building a random forest, at each split in the tree, the algorithm is not even allowed to consider a majority of the available predictors. This may sound crazy, but it has a clever rationale. Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors. Then in the collection of bagged trees, most or all of the trees will use this strong predictor in the top split. Consequently, all of the bagged trees will look quite similar to each other. Hence the predictions from the bagged trees will be highly correlated. Unfortunately, averaging many highly correlated quantities does not lead to as large of a reduction in variance as averaging many uncorrelated quantities. In particular, this means that bagging will not lead to a substantial reduction in variance over a single tree in this setting.

Random forests overcome this problem by forcing each split to consider only a subset of the predictors. Therefore, on average $(p - m)/p$ of the splits will not even consider the strong predictor, and so other predictors will have more of a chance. We can think of this process as decorrelating the trees, thereby making the average of the resulting trees less variable and hence more reliable. The main difference between bagging and random forests is the choice of predictor subset size m. For instance, if a random forest is built using $m = p$, then this amounts simply to bagging. Using a small value of m in building a random forest will typically be helpful when we have a large number of correlated predictors.

Boosting

We are now discussing boosting, yet another approach for improving the predictions resulting from a decision tree. Like bagging, boosting is a general approach that can be applied to many statistical learning methods for regression or classification. Here we restrict our discussion of boosting to the context of decision trees.

Boosting works in a similar way to bagging, except that the trees are grown sequentially: each tree is grown using information from previously grown trees. Boosting does not involve bootstrap sampling; instead, each tree is fit on a modified version of the original data set.

Algorithm 8.2 *Boosting for Regression Trees*

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \ldots, B$, repeat:
   (a) Fit a tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to the training data (X, r).
   (b) Update $\hat{f}$ by adding in a shrunken version of the new tree:
   $$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \tag{8.10}$$
   (c) Update the residuals,
   $$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \tag{8.11}$$
3. Output the boosted model,
   $$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x). \tag{8.12}$$

What is the idea behind this procedure? Unlike fitting a single large decision tree to the data, which amounts to fitting the data hard and potentially overfitting, the boosting approach instead learns slowly. Given the current model, we fit a decision tree to the residuals from the model. That is, we fit a tree using the current residuals, rather than the outcome Y, as the response. We then add this new decision tree into the fitted function in order to update the residuals. Each of these trees can be rather small, with just a few terminal nodes, determined by the parameter d in the algorithm. By fitting small trees to the residuals, we slowly improve $\hat{f}$ in areas where it does not perform well. The shrinkage parameter λ slows the process down even further, allowing more and different shaped trees to attack the residuals. In general, statistical learning approaches that learn slowly tend to perform well. Note that in boosting, unlike in bagging, the construction of each tree depends strongly on the trees that have already been grown.

Boosting has three tuning parameters:

1. The number of trees B. Unlike bagging and random forests, boosting can overfit if B is too large, although this overfitting tends to occur slowly if at all. We use cross-validation to select B.
2. The shrinkage parameter λ, a small positive number. This controls the rate at which boosting learns. Typical values are 0.01 or 0.001, and the right choice can depend on the problem. Very small λ can require using a very large value of B in order to achieve good performance.
3. The number d of splits in each tree, which controls the complexity of the boosted ensemble. Often d = 1 works well, in which case each tree is a stump, consisting of a single split. In this case, the boosted stump ensemble is fitting an additive model,

since each term involves only a single variable. More generally d is the interaction depth, and controls interaction the interaction order of the boosted model, since d splits can involve depth at most d variables.

Summary of tree ensemble methods

Trees are an attractive choice of weak learner for an ensemble method for a number of reasons, including their flexibility and ability to handle predictors of mixed types (i.e. qualitative as well as quantitative). We have now seen four approaches for fitting an ensemble of trees: bagging, random forests, boosting, and BART (not relevant).

- In bagging, the trees are grown independently on random samples of the observations. Consequently, the trees tend to be quite similar to each other. Thus, bagging can get caught in local optima and can fail to thoroughly explore the model space.
- In random forests, the trees are once again grown independently on random samples of the observations. However, each split on each tree is performed using a random subset of the features, thereby decorrelating the trees, and leading to a more thorough exploration of model space relative to bagging.
- In boosting, we only use the original data, and do not draw any random samples. The trees are grown successively, using a "slow" learning approach: each new tree is fit to the signal that is left over from the earlier trees, and shrunken down before it is used.

Chapter 9: Support Vector Machines

Support Vector Machines (SVM) is an approach to classification that has been shown to perform well in a variety of settings and are often considered one of the best "out of the box" classifiers. SVM is a generalization of a simple and intuitive classifier called the maximal margin classifier. People often loosely refer to the maximal margin classifier, the support vector classifier, and the support vector machine as "support vector machines". To avoid confusion, we will carefully distinguish between these three notions in this chapter.

Maximal margin classifier

In this part, we will define a hyperplane and introduce the concept of an optimal separating hyperplane.

What is a hyperplane?

In a p-dimensional space, a hyperplane is a flat affine subspace of dimension p-1. For instance, in two dimensions, a hyperplane is a flat one-dimensional subspace – in other words, a line. In three dimensions, a hyperplane is a flat two-dimensional subspace, that is, a plane. In p > 3 dimensions, it can be hard to visualize a hyperplane, but the notion of a (p − 1)-dimensional flat subspace still applies. The mathematical definition of a hyperplane is quite simple. In two dimensions, a hyperplane is defined by the equation

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$

This equation can easily be extended to a p-dimensional setting:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0$$

This defines a p-dimensional hyperplane, in the sense that if a point $X = \left(X_1, X_2, \dots, X_p\right)^T$ in p-dimensional space (i.e. a vector of length p) satisfies this equation, then X lies on the hyperplane. On the other hand, if the equation amounts to less than or more than zero, X is on one or another side of the hyperplane.

Classification using a separating hyperplane

We suppose that we have a matrix of n training observations in a p-dimensional space that fall into two classes, where -1 represents one class while 1 represents the other. We also have a test observation that we want to classify. Suppose that it is possible to construct a hyperplane that separates the training observations perfectly according to their class labels. Such a separating hyperplane has the property that

$$\beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_p X_{ip} > 0 \; if \; y_i = 1$$

and

$$\beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_p X_{ip} < 0 \; if \; y_i = -1$$

Equivalently, a separating hyperplane has the property that

$$y_i(\beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_p X_{ip}) > 0$$

For all $i = 1, \dots, n$. If a separating hyperplane exists, we can use it to construct a very natural classifier: a test observation is assigned a class depending on which side of the hyperplane it is located. We can also make use of the distance from the newly classified test object to the hyperplane. The further the point is from the hyperplane, the more confident one can be of having classified correctly.

The maximal margin classifier

In general, if our data can be perfectly separated using a hyperplane, then there will in fact exist an infinite number of such hyperplanes. This is because a given separating hyperplane can usually be shifted a tiny bit up or down, or rotated, without coming into contact with any of the observations. In order to construct a classifier based upon a separating hyperplane, we must have a reasonable way to decide which of the infinite possible separating hyperplanes to use.

A natural choice is the maximal margin hyperplane (also known as the optimal separating hyperplane), which is the separating hyperplane that is farthest from the training observations. That is, we can compute the (perpendicular) distance from each training observation to a given separating hyperplane; the smallest such distance is the minimal distance from the observations to the hyperplane and is known as the margin. The maximal margin hyperplane is the separating hyperplane for which the margin is largest, that is, it is the hyperplane that has the farthest minimum distance to the training observations. We can then classify a test observation based on which side of the maximal margin hyperplane it lies. This is known as the maximal margin classifier. We hope that a classifier that has a large margin on the training data will also have a large margin on the test data, and hence will classify the test observations correctly. Although the maximal margin classifier is often successful, it can also lead to overfitting when p is large.

The non-separable case

The maximal margin classifier is a very natural way to perform classification, if a separating hyperplane exists. However, in many cases no separating hyperplane exists, and so there is no maximal margin classifier. In this case, the optimization problem (9.9)–(9.11) has no solution with M >0. An example is shown in Figure 9.4. In this case, we cannot exactly separate the two classes. However, as we will see in the next section, we can extend the concept of a separating hyperplane in order to develop a hyperplane that almost separates the classes, using a so-called soft margin. The generalization of the maximal margin classifier to the non-separable case is known as the support vector classifier.

Support vector classifiers

Observations that belong to two classes are not necessarily separable by a hyperplane. In fact, even if a separating hyperplane does exist, then there are instances in which a classifier based on a separating hyperplane might not be desirable. A classifier based on a separating hyperplane will necessarily perfectly classify all of the training observations; this can lead to sensitivity to individual observations. The introduction of a single observation can lead to a dramatic change in the maximal margin hyperplane. If this hyperplane is very sensitive to such changes in the data, it is most likely overfit.

In such cases, it could be worthwhile to misclassify a few training observations in order to do a better job in classifying the remaining observations. The support vector classifier, sometimes called a soft margin classifier, does exactly this. Rather than seeking the largest possible margin so that every observation is not only on the correct side of the hyperplane but also on the correct side of the margin, we instead allow some observations to be on the incorrect side of the margin, or even the incorrect side of the hyperplane. (The margin is soft because it can be violated by some of the training observations.) An observation can be not only on the wrong side of the margin, but also on the wrong side of the hyperplane. In fact, when there is no separating hyperplane, such a situation is inevitable. Observations on the wrong side of the hyperplane correspond to training observations that are misclassified by the support vector classifier.

Details of the support vector classifier

The support vector classifier classifies a test observation depending on which side of a hyperplane it lies. The hyperplane is chosen to correctly separate most of the training observations into the two classes but may misclassify a few observations. It is the solution to the optimization problem

$$\underset{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n,\,M}{\text{maximize}} \quad M \tag{9.12}$$

$$\text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 = 1, \tag{9.13}$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) \geq M(1 - \epsilon_i), \tag{9.14}$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^{n} \epsilon_i \leq C, \tag{9.15}$$

where C is a nonnegative tuning parameter. As in (9.11), M is the width of the margin; we seek to make this quantity as large as possible. In (9.14), $\epsilon_1, \ldots, \epsilon_n$ are slack variables that

allow individual observations to be on slack the wrong side of the margin or the hyperplane; we will explain them in variable greater detail momentarily. Once we have solved (9.12)–(9.15), we classify a test observation x^* as before, by simply determining on which side of the hyperplane it lies. That is, we classify the test observation based on the sign of $f(x^*) = \beta_0 + \beta_1 x_1^* + \cdots + \beta_p x_p^*$. The problem (9.12)–(9.15) seems complex, but insight into its behavior can be made through a series of simple observations presented below. First of all, the slack variable ϵ_i tells us where the ith observation is located, relative to the hyperplane and relative to the margin. If $\epsilon_i = 0$ then the ith observation is on the correct side of the margin, as we saw in Section 9.1.4. If $\epsilon_i > 0$ then the ith observation is on the wrong side of the margin, and we say that the ith observation has violated the margin. If $\epsilon_i > 1$ then it is on the wrong side of the hyperplane.

We now consider the role of the tuning parameter C. In (9.15), C bounds the sum of the ϵ_i's, and so it determines the number and severity of the violations to the margin (and to the hyperplane) that we will tolerate. We can think of C as a budget for the amount that the margin can be violated by the n observations. If C = 0 then there is no budget for violations to the margin, and it must be the case that $\epsilon_1 = \cdots = \epsilon_n = 0$, in which case (9.12)–(9.15) simply amounts to the maximal margin hyperplane optimization problem (9.9)–(9.11). (Of course, a maximal margin hyperplane exists only if the two classes are separable.) For C > 0 no more than C observations can be on the wrong side of the hyperplane, because if an observation is on the wrong side of the hyperplane then $\epsilon_i > 1$, and (9.15) requires that $\sum_{i=1}^{n} \epsilon_i \leq C$. As the budget C increases, we become more tolerant of violations to the margin, and so the margin will widen. Conversely, as C decreases, we become less tolerant of violations to the margin and so the margin narrows.

In practice, C is treated as a tuning parameter that is generally chosen via cross-validation. As with the tuning parameters that we have seen throughout this book, C controls the bias-variance trade-off of the statistical learning technique. When C is small, we seek narrow margins that are rarely violated; this amounts to a classifier that is highly fit to the data, which may have low bias but high variance. On the other hand, when C is larger, the margin is wider and we allow more violations to it; this amounts to fitting the data less hard and obtaining a classifier that is potentially more biased but may have lower variance.

The optimization problem (9.12)–(9.15) has a very interesting property: it turns out that only observations that either lie on the margin or that violate the margin will affect the hyperplane, and hence the classifier obtained. In other words, an observation that lies strictly on the correct side of the margin does not affect the support vector classifier! Changing the position of that observation would not change the classifier at all, provided that its position remains on the correct side of the margin. Observations that lie directly on the margin, or on the wrong side of the margin for their class, are known as support vectors. These observations do affect the support vector classifier.

The fact that only support vectors affect the classifier is in line with our previous assertion that C controls the bias-variance trade-off of the support vector classifier. When the tuning parameter C is large, then the margin is wide, many observations violate the margin, and so there are many support vectors. In this case, many observations are involved in determining the hyperplane. The top left panel in Figure 9.7 illustrates this setting: this classifier has low variance (since many observations are support vectors) but potentially high bias. In contrast, if C is small, then there will be fewer support vectors and hence the resulting classifier will have low bias but high variance.

The fact that the support vector classifier's decision rule is based only on a potentially small subset of the training observations (the support vectors) means that it is quite robust to the behavior of observations that are far away from the hyperplane. This property is distinct from some of the other classification methods that we have seen in preceding chapters, such as linear discriminant analysis. Recall that the LDA classification rule depends on the mean of all of the observations within each class, as well as the within-class covariance matrix computed using all of the observations. In contrast, logistic regression, unlike LDA, has very low sensitivity to observations far from the decision boundary. In fact we will see in Section 9.5 that the support vector classifier and logistic regression are closely related.

Support vector machines

We first discuss a general mechanism for converting a linear classifier into one that produces non-linear decision boundaries. We then introduce the support vector machine, which does this in an automatic way.

Classification with non-linear decision boundaries

The support vector classifier is a natural approach for classification in the two-class setting, if the boundary between the two classes is linear. However, in practice we are sometimes faced with non-linear class boundaries.

In Chapter 7, we are faced with an analogous situation. We see there that the performance of linear regression can suffer when there is a nonlinear relationship between the predictors and the outcome. In that case, we consider enlarging the feature space using functions of the predictors, such as quadratic and cubic terms, in order to address this non-linearity. In the case of the support vector classifier, we could address the problem of possibly non-linear boundaries between classes in a similar way, by enlarging the feature space using quadratic, cubic, and even higher-order polynomial functions of the predictors. For instance, rather than fitting a support vector classifier using p features

$$X_1, X_2, \dots, X_p$$

We could instead fit a support vector classifier using 2p features

$$X_1, X_1^2, X_2, X_2^2, \dots, X_p, X_p^2$$

Then, (9.12)-(9.15) would become

$$\underset{\beta_0,\beta_{11},\beta_{12},\dots,\beta_{p1},\beta_{p2},\epsilon_1,\dots,\epsilon_n,\,M}{\text{maximize}} \quad M \tag{9.16}$$

$$\text{subject to } y_i\left(\beta_0 + \sum_{j=1}^{p}\beta_{j1}x_{ij} + \sum_{j=1}^{p}\beta_{j2}x_{ij}^2\right) \geq M(1-\epsilon_i),$$

$$\sum_{i=1}^{n}\epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \sum_{j=1}^{p}\sum_{k=1}^{2}\beta_{jk}^2 = 1.$$

Why does this lead to a non-linear decision boundary? In the enlarged feature space, the decision boundary that results from (9.16) is in fact linear. But in the original feature space, the decision boundary is of the form q(x) = 0, where q is a quadratic polynomial, and its solutions are generally non-linear. One might additionally want to enlarge the feature space with higher-order polynomial terms, or with interaction terms of the form $X_jX_j, for\ j \neq j'$.

Alternatively, other functions of the predictors could be considered rather than polynomials. It is not hard to see that there are many possible ways to enlarge the feature space, and that unless we are careful, we could end up with a huge number of features. Then computations would become unmanageable. The support vector machine, which we present next, allows us to enlarge the feature space used by the support vector classifier in a way that leads to efficient computations.

The support vector machine

The support vector machine (SVM) is an extension of the support vector classifier that results from enlarging the feature space in a specific way, using kernels. We will now discuss this extension, the details of which are somewhat complex and beyond the scope of this book. However, the main kernel idea is described in Section 9.3.1: we may want to enlarge our feature space in order to accommodate a non-linear boundary between the classes. The kernel approach that we describe here is simply an efficient computational approach for enacting this idea.

In statistical learning, a kernel is a function that implicitly maps data into a higher-dimensional space without explicitly computing the transformation, allowing algorithms to capture nonlinear relationships using linear methods in that transformed space. Kernels are crucial in methods like Support Vector Machines (SVM) and kernel PCA, enabling them to model complex, nonlinear patterns while maintaining computational efficiency. They allow linear algorithms to operate in rich, flexible feature spaces through inner products. Think of trying to separate points shaped like a circle in 2D. In the original space, no straight line can do it. A kernel acts like lifting the data into 3D, where a flat plane can separate them. You don't need to know the full transformation, just the effect of the transformation on pairs of points, which the kernel provides directly.

Two commonly used non-linear kernels are the polynomial kernel and the radial kernel16....

Polynomial Kernel: This kernel takes the form $K(x_i, x_i') = \left(1 + \sum_{j=i}^{p} x_{ij} x_i' j\right)^d$, where d is a positive integer called the degree. Using a polynomial kernel with degree d > 1 allows for a much more flexible decision boundary than the standard linear kernel (which is equivalent to a polynomial kernel with d = 1. It is essentially like fitting a support vector classifier in a higher-dimensional space that includes polynomial terms up to degree.

Radial Kernel: This kernel takes the form $K(x_i, x_i') = \exp\left(-\gamma \sum_{j=i}^{p} \left(x_{ij} x_i' j\right)^2\right)$. Here, γ is a positive constant. The radial kernel quantifies similarity based on the Euclidean distance between observations. If two observations are far apart in the original feature space (large Euclidean distance), the value of the kernel K will be very small. This means that training observations far from a new test observation will have very little influence on its predicted class label. Consequently, the radial kernel is described as having very local behavior, where only nearby training observations significantly affect a test observation's classification.

Using kernels allows SVMs to fit complex non-linear decision boundaries without explicitly computing the potentially infinite-dimensional feature space, making the computations efficient. For some kernels, like the radial kernel, the feature space is implicitly infinite-dimensional. Applying SVMs with a radial kernel using different values of γ can result in varying degrees of non-linearity in the fit.

Chapter 12: Unsupervised learning

This chapter will focus on unsupervised learning, a set of statistical tools intended for the setting in which we have only a set of features $X_1, X_2, \ldots, X_p$ measured on n observations. We are not interested in prediction, because we do not have an associated response variable Y. Rather, the goal is to discover interesting things about the measurements on $X_1, X_2, \ldots, X_p$. Is there an informative way to visualize the data? Can we discover subgroups among the variables or among the observations? Unsupervised learning refers to a diverse set of techniques for answering questions such as these. In this chapter, we will focus on two particular types of unsupervised learning: principal components analysis, a tool used for data visualization or data pre-processing before supervised techniques are applied, and clustering, a broad class of methods for discovering unknown subgroups in data.

The challenge of unsupervised learning

Unsupervised learning is often performed as part of an exploratory data analysis. Furthermore, it can be hard to assess the results obtained from unsupervised learning methods, since there is no universally accepted mechanism for performing cross-validation or validating results on an independent data set. The reason for this difference is simple. If we fit a predictive model using a supervised learning technique, then it is possible to check our work by seeing how well our model predicts the response Y on observations not used in fitting the model. However, in unsupervised learning, there is no way to check our work because we don't know the true answer, the problem is unsupervised.

Principal components analysis

When faced with a large set of correlated variables, principal components allow us to summarize this set with a smaller number of representative variables that collectively explain most of the variability in the original set.

Principal components analysis (PCA) refers to the process by which principal components are computed, and the subsequent use of these components in understanding the data. PCA is an unsupervised approach, since it involves only a set of features $X_1, X_2, \ldots, X_p$, and no associated response Y. Apart from producing derived variables for use in supervised learning problems, PCA also serves as a tool for data visualization (visualization of the observations or visualization of the variables). It can also be used as a tool for data imputation — that is, for filling in missing values in a data matrix.

What are principal components?

Suppose that we wish to visualize n observations with measurements on a set of p features, $X_1, X_2, \ldots, X_p$, as part of an exploratory data analysis. We could do this by examining two-dimensional scatterplots of the data, each of which contains the n observations' measurements on two of the features. However, there are $\binom{p}{2} = p(p-1)/2$ such scatterplots; for example, with p = 10 there are 45 plots! If p is large, then it will certainly not be possible to look at all of them; moreover, most likely none of them will be informative since they each contain just a small fraction of the total information present in the data set. Clearly, a better method is required to visualize the n observations when p is large. In particular, we would like to find a low-dimensional representation of the data that captures as much of the information as possible. For instance, if we can obtain a two-dimensional

representation of the data that captures most of the information, then we can plot the observations in this low-dimensional space.

PCA provides a tool to do just this. It finds a low-dimensional representation of a data set that contains as much as possible of the variation. The idea is that each of the n observations lives in p-dimensional space, but not all of these dimensions are equally interesting. PCA seeks a small number of dimensions that are as interesting as possible, where the concept of interesting is measured by the amount that the observations vary along each dimension. Each of the dimensions found by PCA is a linear combination of the p features. We now explain the manner in which these dimensions, or principal components, are found.

The first principal component of a set of features $X_1, X_2, \dots, X_p$ is the normalized linear combination of the features

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \cdots + \phi_{p1}X_p$$

That has the largest variance. By normalized, we mean that $\sum_{j=1}^{p} \phi_{j1}^2 = 1$. We refer to the elements $\phi_{11}, \dots, \phi_{p1}$ as the loadings of the first principal component; together, the loadings make up the principal component loading vector, $\phi_1 = \left(\phi_{11}\phi_{21} \dots \phi_{p1}\right)^T$. We constrain the loadings so that their sum of squares is equal to one, since otherwise setting these elements to be arbitrarily large in absolute value could result in arbitrarily large variance.

Given an n × p data set X, how do we compute the first principal component? Since we are only interested in variance, we assume that each of the variables in X has been centered to have mean zero (that is, the column means of X are zero). We then look for the linear combination of the sample feature values of the form

$$z_{i1} = \phi_{11}x_{i1} + \phi_{21}x_{i2} + \cdots + \phi_{p1}x_{ip}$$

That has the largest sample variance, subject to the constraint that $\sum_{j=1}^{p} \phi_{j1}^2 = 1$. In other words, the first principal component loading vector solves the optimization problem

$$\underset{\phi_{11},\dots,\phi_{p1}}{\text{maximize}} \left\{ \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{p} \phi_{j1} x_{ij} \right)^2 \right\} \text{ subject to } \sum_{j=1}^{p} \phi_{j1}^2 = 1. \qquad (12.3)$$

There is a nice geometric interpretation of the first principal component. The loading vector ϕ_1 with elements $\phi_{11}, \phi_{21}, \dots, \phi_{p1}$ defines a direction in feature space along which the data vary the most. If we project the n data points $x_1, \dots, x_n$ onto this direction, the projected values are the principal component scores $z_{11}, \dots, z_{n1}$ themselves.

After the first principal component $Z_1$of the features has been determined, we can find the second principal component Z_2. The second principal component is the linear combination of $X_1, \dots, X_p$ that has maximal variance out of all linear combinations that are uncorrelated with Z_1. The second principal component scores $z_{12}, z_{22}, \dots, z_{n2}$ take the form

$$z_{i2} = \phi_{12}x_{i1} + \phi_{22}x_{i2} + \cdots + \phi_{p2}x_{ip}$$

where $\phi_2$is the second principal component loading vector, with elements $\phi_{12}, \phi_{22}, \dots, \phi_{p2}$. It turns out that constraining Z_2 to be uncorrelated with Z_1 is equivalent to constraining the direction ϕ_2 to be orthogonal (perpendicular) to the direction ϕ_1.

Once we have computed the principal components, we can plot them against each other in order to produce low-dimensional views of the data. For instance, we can plot the score vector Z_1 against Z_2, Z_1against Z_3, Z_2 against Z_3, and so forth. Geometrically, this amounts to projecting the original data down onto the subspace spanned by ϕ_1, ϕ_2, and ϕ_3, and plotting the projected points.

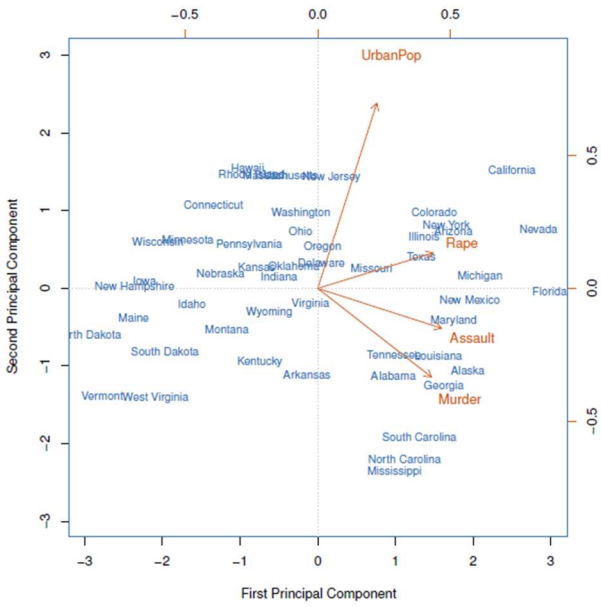


FIGURE 12.1. *The first two principal components for the* USArrests *data. The blue state names represent the scores for the first two principal components. The orange arrows indicate the first two principal component loading vectors (with axes on the top and right). For example, the loading for* Rape *on the first component is* 0.54, *and its loading on the second principal component* 0.17 *(the word* Rape *is centered at the point* $(0.54, 0.17)$*). This figure is known as a biplot, because it displays both the principal component scores and the principal component loadings.*

Clustering methods

Clustering refers to a very broad set of techniques for finding subgroups, or clusters, in a data set. When we cluster the observations of a data set, we seek to partition them into distinct groups so that the observations within each group are quite similar to each other, while observations in different groups are quite different from each other. Of course, to make this concrete, we must define what it means for two or more observations to be similar or different. Indeed, this is often a domain-specific consideration that must be made based on knowledge of the data being studied.

Both clustering and PCA seek to simplify the data via a small number of summaries, but their mechanisms are different:

- PCA looks to find a low-dimensional representation of the observations that explain a good fraction of the variance
- Clustering looks to find homogenous subgroups among the observations

Since clustering is popular in many fields, there exist a great number of clustering methods. In this section we focus on perhaps the two best-known clustering approaches: K-means clustering and hierarchical clustering. In K-means clustering, we seek to partition the observations into a pre-specified number of clusters. On the other hand, in hierarchical

clustering, we do not know in advance how many clusters we want; in fact, we end up with a tree-like visual representation of the observations, called a dendrogram, that allows us to view at once the clusterings obtained for each possible number of clusters, from 1 to n. There are advantages and disadvantages to each of these clustering approaches, which we highlight in this chapter.

In general, we can cluster observations on the basis of the features in order to identify subgroups among the observations, or we can cluster features on the basis of the observations in order to discover subgroups among the features. In what follows, for simplicity we will discuss clustering observations on the basis of the features, though the converse can be performed by simply transposing the data matrix.

K-means clustering

K-means clustering is a simple and elegant approach for partitioning a data set into K distinct, non-overlapping clusters. To perform K-means clustering, we must first specify the desired number of clusters K; then the K-means algorithm will assign each observation to exactly one of the K clusters.

The K-means clustering procedure results from a simple and intuitive mathematical problem. We begin by defining some notation. Let $C_1, \ldots, C_K$ denote sets containing the indices of the observations in each cluster. These sets satisfy two properties:

1. $C_1 \cup C_2 \cup \ldots \cup C_K = \{1, \ldots, n\}$. In other words, each observation belongs to at least one of the clusters.
2. $C_k \cap C_{k'} = \emptyset$ for all $k \neq k'$. In other words, the clusters are non-overlapping: no observation belongs to more than one cluster

For instance, if the ith observation is in the kth cluster, then $i \in C_k$. The idea behind K-means clustering is that a good clustering is one for which the within-cluster variation is as small as possible. The within-cluster variation for cluster C_k is a measure $W(C_k)$ of the amount by which the observations within a cluster differ from each other. Hence we want to solve the problem

$$\underset{C_1,\ldots,C_K}{\text{minimize}} \left\{ \sum_{k=1}^{K} W(C_k) \right\}. \qquad (12.15)$$

In words, this formula says that we want to partition the observations into K clusters such that the total within-cluster variation, summed over all K clusters, is as small as possible. Solving (12.15) seems like a reasonable idea, but in order to make it actionable we need to define the within-cluster variation. There are many possible ways to define this concept, but by far the most common choice involves squared Euclidean distance. That is, we define

$$W(C_k) = \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2, \qquad (12.16)$$

where $|C_k|$ denotes the number of observations in the kth cluster. In other words, the within-cluster variation for the kth cluster is the sum of all of the pairwise squared Euclidean distances between the observations in the kth cluster, divided by the total number of observations in the kth cluster. Combining (12.15) and (12.16) gives the optimization problem that defines K-means clustering,

$$\underset{C_1,\ldots,C_K}{\text{minimize}} \left\{ \sum_{k=1}^{K} \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 \right\}. \qquad (12.17)$$

Now, we would like to find an algorithm to solve (12.17)—that is, a method to partition the observations into K clusters such that the objective of (12.17) is minimized. This is in fact a very difficult problem to solve precisely, since there are almost K^n ways to partition n observations into K clusters. This is a huge number unless K and n are tiny! Fortunately, a very simple algorithm can be shown to provide a local optimum—a pretty good solution—to the K-means optimizat/ion problem (12.17). This approach is laid out in Algorithm 12.2.

Algorithm 12.2 *K-Means Clustering*

1. Randomly assign a number, from 1 to K, to each of the observations. These serve as initial cluster assignments for the observations.
2. Iterate until the cluster assignments stop changing:
   (a) For each of the K clusters, compute the cluster *centroid*. The kth cluster centroid is the vector of the p feature means for the observations in the kth cluster.
   (b) Assign each observation to the cluster whose centroid is closest (where *closest* is defined using Euclidean distance).

In Step 2(a) the cluster means for each feature are the constants that minimize the sum-of-squared deviations, and in Step 2(b), reallocating the observations can only improve (12.18). This means that as the algorithm is run, the clustering obtained will continually improve until the result no longer changes; the objective of (12.17) will never increase. When the result no longer changes, a local optimum has been reached. Figure 12.8 shows the progression of the algorithm on the toy example from Figure 12.7. K-means clustering derives its name from the fact that in Step 2(a), the cluster centroids are computed as the mean of the observations assigned to each cluster.

Because the K-means algorithm finds a local rather than a global optimum, the results obtained will depend on the initial (random) cluster assignment of each observation in Step 1 of Algorithm 12.2. For this reason, it is important to run the algorithm multiple times from different random initial configurations. Then one selects the best solution, i.e. that for which the objective (12.17) is smallest.

Hierarchical clustering

One potential disadvantage of K-means clustering is that it requires us to pre-specify the number of clusters K. Hierarchical clustering is an alternative approach which does not require that we commit to a particular choice of K. Hierarchical clustering has an added advantage over K-means clustering in that it results in an attractive tree-based representation of the observations, called a dendrogram.

In this section, we describe bottom-up or agglomerative clustering. This is the most common type of hierarchical clustering and refers to the fact that a dendrogram generally is built starting from the leaves and combining clusters up to the trunk.

Interpreting a dendrogram

Each leaf of a dendrogram represents the individual observations in the data. However, as we move up the tree, some leaves begin to fuse into branches. These correspond to observations that are similar to each other. As we move higher up the tree, branches themselves fuse, either with leaves or other branches. The earlier (lower in the tree) fusions occur, the more similar the groups of observations are to each other. On the other hand, observations that fuse later (near the top of the tree) can be quite different. In fact, this statement can be made precise: for any two observations, we can look for the point in the tree where branches containing those two observations are first fused. The height of this fusion, as measured on the vertical axis, indicates how different the two observations are. Thus, observations that fuse at the very bottom of the tree are quite similar to each other, whereas observations that fuse close to the top of the tree will tend to be quite different.

We cannot draw conclusions about the similarity of two observations based on their proximity along the horizontal axis. Rather, we draw conclusions about the similarity of two observations based on the location on the vertical axis where branches containing those two observations first are fused. The height of the cut to the dendrogram serves the same role as the K in K-means clustering: it controls the number of clusters obtained.

The hierarchical clustering algorithm

The hierarchical clustering dendrogram is obtained via an extremely simple algorithm. We begin by defining some sort of dissimilarity measure between each pair of observations. Most often, Euclidean distance is used; we will discuss the choice of dissimilarity measure later in this chapter. The algorithm proceeds iteratively. Starting out at the bottom of the dendrogram, each of the n observations is treated as its own cluster. The two clusters that are most similar to each other are then fused so that there now are n−1 clusters. Next the two clusters that are most similar to each other are fused again, so that there now are n − 2 clusters. The algorithm proceeds in this fashion until all of the observations belong to one single cluster, and the dendrogram is complete.

We have a concept of the dissimilarity between pairs of observations, but how do we define the dissimilarity between two clusters if one or both of the clusters contains multiple observations? The concept of dissimilarity between a pair of observations needs to be extended to a pair of groups of observations. This extension is achieved by developing the notion of linkage, which defines the dissimilarity between two groups of observations. The four most common types of linkage—complete, average, single, and centroid—are briefly described in Table 12.3

Algorithm 12.3 *Hierarchical Clustering*

1. Begin with n observations and a measure (such as Euclidean distance) of all the $\binom{n}{2} = n(n-1)/2$ pairwise dissimilarities. Treat each observation as its own cluster.
2. For $i = n, n-1, \ldots, 2$:
   (a) Examine all pairwise inter-cluster dissimilarities among the i clusters and identify the pair of clusters that are least dissimilar (that is, most similar). Fuse these two clusters. The dissimilarity between these two clusters indicates the height in the dendrogram at which the fusion should be placed.
   (b) Compute the new pairwise inter-cluster dissimilarities among the $i-1$ remaining clusters.

Linkage	*Description*
Complete	Maximal intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the *largest* of these dissimilarities.
Single	Minimal intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the *smallest* of these dissimilarities. Single linkage can result in extended, trailing clusters in which single observations are fused one-at-a-time.
Average	Mean intercluster dissimilarity. Compute all pairwise dissimilarities between the observations in cluster A and the observations in cluster B, and record the *average* of these dissimilarities.
Centroid	Dissimilarity between the centroid for cluster A (a mean vector of length p) and the centroid for cluster B. Centroid linkage can result in undesirable *inversions*.

TABLE 12.3. *A summary of the four most commonly-used types of linkage in hierarchical clustering.*

Average, complete and single linkage are most popular among statisticians. Average and complete linkage are generally preferred over single linkage, as they tend to yield more balanced dendrograms. Centroid linkage is often used in genomics, but suffers from a major drawback in that an inversion can occur, whereby two clusters are fused at a height below either of the individual clusters in the dendrogram. This can lead to difficulties in visualization as well as in interpretation of the dendrogram. The dissimilarities computed in Step 2(b) of the hierarchical clustering algorithm will depend on the type of linkage used, as well as the choice of dissimilarity measure. Hence, the resulting dendrogram typically depends quite strongly on the type of linkage used.

Choice of dissimilarity measure

Thus far, the examples in this chapter have used Euclidean distance as the dissimilarity measure. But sometimes other dissimilarity measures might be preferred. For example, correlation-based distance considers two observations to be similar if their features are highly correlated, even though the observed values may be far apart in terms of Euclidean distance. This is an unusual use of correlation, which is normally computed between variables; here it is computed between the observation profiles for each pair of observations. Correlation-based distance focuses on the shapes of observation profiles rather than their magnitudes.

The choice of dissimilarity measure is very important, as it has a strong effect on the resulting dendrogram. In general, careful attention should be paid to the type of data being

clustered and the scientific question at hand. These considerations should determine what type of dissimilarity measure is used for hierarchical clustering.

In addition to carefully selecting the dissimilarity measure used, one must also consider whether or not the variables should be scaled to have standard deviation one before the dissimilarity between the observations is computed.